UNIVERSITY OF AMSTERDAM
Faculty of Science
MSc Computer Science *(joint degree)*

Master's Thesis

Splatt3R-SLAM: From Image Pairs to Persistent Maps

Zelong Li
loong.li2@student.uva.nl
+31 6 3023 2998
Student number 15326322

Supervisor:	Prof. Martin Oswald
Daily supervisor:	Dr. Qi Zhang
Second reader:	*[to be assigned — fill in]*

Submitted in partial fulfilment of the requirements
for the degree of Master of Science

October 15, 2026

Abstract

An image pair can now produce a dense 3D Gaussian reconstruction in one network pass. A video instead creates overlapping predictions that must follow loop-closure corrections. This thesis presents Splatt3R-SLAM, a monocular system that turns local predictions into a persistent map. It gives each local Gaussian map and its observation cameras the same keyframe anchor, so pose corrections move both together. A separate process refines appearance while MASt3R-SLAM estimates motion.

Head-only adaptation changes prediction, opacity attenuation changes insertion, and photometric refinement uses sequence observations. Refinement improves PSNR by 1.00–8.69 dB across four controls. Attenuation improves LPIPS in ten of twelve tests, although confidence weighting offers little advantage over uniform attenuation.

The released head with 300 s refinement achieves state-of-the-art performance. On eight Replica scenes, it reaches 23.02 dB mean PSNR versus 19.48 dB for Photo-SLAM, with wins on every scene. On TUM desk, its 13.95 dB is close to MonoGS. Dense prediction provides a useful initialisation but creates millions of primitives. Compact online mapping remains the main open problem.

Acknowledgements

[This page is a placeholder to be personalised before submission.]

I would like to thank my supervisors and second reader for their guidance. I also thank the authors and maintainers of MASt3R-SLAM, Splatt3R, MonoGS, Photo-SLAM, and VGGT-SLAM for making their research code available.

Contents

Abstract i

Acknowledgements ii

Notation and Abbreviations ix

1 Introduction 1
- 1.1 The central idea . . . 1
- 1.2 Research questions . . . 2
- 1.3 From design to evidence . . . 2
- 1.4 Contributions . . . 3
- 1.5 Thesis structure . . . 3

2 Background 4
- 2.1 3D Gaussian splatting . . . 4
  - 2.1.1 The representation . . . 4
  - 2.1.2 Projection and rasterisation . . . 4
  - 2.1.3 Background and visibility . . . 5
  - 2.1.4 Per-scene optimisation, and what a feed-forward model skips . . . 5
- 2.2 Feed-forward pointmaps and Gaussians . . . 6
  - 2.2.1 DUSt3R and MASt3R . . . 6
  - 2.2.2 Many-view feed-forward models . . . 6
  - 2.2.3 Splatt3R . . . 6
- 2.3 SLAM with a pointmap prior . . . 7
  - 2.3.1 Poses on the Sim(3) manifold . . . 7
  - 2.3.2 Matching and the ray error . . . 8
  - 2.3.3 Keyframing and canonical pointmap fusion . . . 8
  - 2.3.4 Loop closure and the back-end . . . 9
- 2.4 Metrics . . . 9
  - 2.4.1 Image metrics . . . 9
  - 2.4.2 Trajectory metrics . . . 9

3 Related Work 10
- 3.1 Feed-forward 3D reconstruction . . . 10
- 3.2 Neural and Gaussian map representations . . . 10

3.3 Gaussian-splatting SLAM . . . 10
3.4 Feed-forward priors inside SLAM . . . 11
3.5 Position of this work . . . 11
3.6 Parameter-efficient adaptation . . . 11
3.7 Evaluation practice . . . 11

4 System Design **12**
4.1 Overview . . . 12
4.2 Anchor-carried map composition . . . 12
4.3 The injection pipeline . . . 13
4.4 Rendering, display and the evaluation artefact . . . 14
4.5 Changes to MASt3R-SLAM . . . 14

5 Adapting the Feed-Forward Head **16**
5.1 Choosing the trainable module . . . 16
5.2 Training setup . . . 16
5.3 Photometric augmentation and the opacity channel . . . 17

6 Injection-Time Thinning and the Opacity Mechanism **19**
6.1 The crowding problem . . . 19
6.2 Insertion-time opacity attenuation . . . 19
6.2.1 Relation to pairwise training . . . 20
6.3 Opacity, colour, and visibility . . . 20
6.4 Controls . . . 20
6.4.1 Confidence weighting . . . 20
6.4.2 Shape-weighted attenuation . . . 21

7 Online Map Refinement **22**
7.1 Design . . . 22
7.1.1 Parameterisation . . . 22
7.1.2 Objective . . . 22
7.1.3 Anchor-carried supervision . . . 22
7.1.4 Sampling . . . 23
7.1.5 Scheduling . . . 23
7.1.6 Centre updates . . . 23
7.2 The polish phase . . . 23
7.3 Density control . . . 24
7.4 Tracking preservation . . . 24

8 Evaluation Protocol **25**
8.1 Common rendering conditions . . . 25
8.1.1 Evaluation scope . . . 25
8.1.2 Datasets and configurations . . . 26
8.1.3 Alignment . . . 26
8.2 Self-consistency and provenance . . . 26
8.3 Pose-source sensitivity . . . 27
8.4 Systems compared . . . 27

9 Results **28**
9.1 Head-only adaptation . . . 28
9.1.1 Tested explanations . . . 28
9.2 Injection-time thinning . . . 28
9.2.1 Dose–response . . . 29
9.3 Opacity and input luminance . . . 29
9.4 State-of-the-art performance . . . 29
9.4.1 Scene aggregates and per-frame distributions . . . 30
9.5 Rendering: TUM desk and optimisation budget . . . 31
9.6 Trajectory accuracy . . . 32
9.7 Shared anchors and refinement . . . 34
9.7.1 Pose-correction control . . . 34
9.7.2 Refinement on and off . . . 34
9.8 Online delivery across dataset families . . . 34
9.8.1 Budget and centre-update diagnostics . . . 35
9.9 System cost . . . 36

10 Negative Results **37**
10.1 Backbone adaptation . . . 37
10.2 Independent multi-pair averaging . . . 37
10.3 Loop-closure retrieval refitting . . . 37
10.4 Cross-family adaptation variation . . . 38
10.5 Geometric scale correction . . . 38
10.5.1 Context and backbone . . . 38
10.5.2 Map edit . . . 38
10.5.3 Scale prior during refinement . . . 39
10.6 Result of the negative studies . . . 39

11 Discussion and Limitations **40**
11.1 What shared anchors solve . . . 40
11.2 Map size . . . 40

11.3 Online refinement . . . 40
11.4 Evaluation protocol . . . 41
11.5 Statistical coverage . . . 41

12 Conclusion and Future Work **42**
12.1 Answers to the research questions . . . 42
12.2 Future work . . . 43

Bibliography **44**

A Hyper-parameters and Configuration **47**
A.1 Head-only adaptation . . . 47
A.2 Injection-time thinning . . . 47
A.3 Online refinement . . . 48
A.4 Hardware . . . 48

B Reproduction **49**
B.1 Environment . . . 49
B.2 Running the system . . . 50
B.3 Scoring a map . . . 50
B.4 The self-consistency check . . . 50
B.5 Figures and independently formatted documents . . . 51

C External Baseline Build Records **52**
C.1 Traps common to all four . . . 52
C.2 MASt3R-SLAM . . . 52
C.3 MonoGS . . . 53
C.4 Photo-SLAM . . . 53
C.5 VGGT-SLAM . . . 53

D Extended Result Tables **54**
D.1 Per-family system cost . . . 54
D.2 Compactness curve with LPIPS . . . 54
D.3 Gaussian counts in the Replica comparison . . . 55
D.4 Scale-consistency measurement, per sequence . . . 55
D.5 Multi-view scale audit . . . 55

List of Figures

1.1 Image pairs become a persistent map 2

4.1 System overview 13

9.1 Replica qualitative comparison 31
9.2 Beyond Photo-SLAM 32
9.3 Real reconstructions across datasets 33

List of Tables

8.1 Systems compared, and what each can produce 25

9.1 Paired head-adaptation examples 29
9.2 Four falsified attributions 29
9.3 Fine-grained opacity attenuation ablation 30
9.4 Opacity–luminance correlations on five heads 30
9.5 Rendering on Replica, all eight scenes 31
9.6 Per-frame PSNR differences 32
9.7 Rendering on TUM `fr1_desk` 33
9.8 Trajectory accuracy, TUM freiburg1, five systems 34
9.9 Refinement ablations 35
9.10 Online delivery, GUI on, dual GPU 35
9.11 Refinement iteration-budget control 36
9.12 System cost, all four systems 36

10.1 Adjacent-view depth-scale disagreement 38

11.1 PSNR versus map budget, four Replica scenes 41

Notation and Abbreviations

Symbol	Meaning
I_i	RGB image i; $\mathcal{C}_i$ is its camera frame
$X_{\mathcal{C}_a}^{(b)}$	pointmap of image b, expressed in frame $\mathcal{C}_a$
T_{AB}	transform that maps coordinates from frame B to frame A
$T_{W\mathcal{C}_k}$	camera-to-world pose of keyframe k
a_k, R_k, t_k	similarity scale, rotation, and translation of keyframe k
μ_n, Σ_n, s_n	centre, covariance, and three axis scales of Gaussian n
$\gamma_n, \hat{\gamma}_n$	pointmap confidence and its within-keyframe rank
$\omega_{m,n}$	matching confidence for correspondence (m, n)
α_n	activated opacity of Gaussian n
α'_n	opacity after injection-time thinning
λ	thinning strength, `--refiner-conf-fade`
λ_D	D-SSIM weight in the refinement objective
$\mathcal{L}_{\text{ref}}$	online refinement objective
$\hat{I}$	rendered image

Abbrev.	Expansion
3DGS	3D Gaussian Splatting
ATE	Absolute Trajectory Error
D-SSIM	structural dissimilarity, $1 - \mathrm{SSIM}$
GS	Gaussian Splatting
LPIPS	Learned Perceptual Image Patch Similarity
LoRA	Low-Rank Adaptation
PSNR	Peak Signal-to-Noise Ratio
RMSE	Root-Mean-Square Error
SLAM	Simultaneous Localisation and Mapping
SSIM	Structural Similarity Index Measure

Chapter 1

Introduction

A robot map must remain useful after the camera has left a room. It must support localisation, preserve scene structure, and show what the scene looks like from another viewpoint. Classical visual SLAM provides camera motion and geometry. 3D Gaussian splatting [1] adds a renderable appearance model.

Most Gaussian SLAM systems build this model through per-scene optimisation. MonoGS [2], Photo-SLAM [3], and related systems create and update primitives as images arrive. Feed-forward reconstruction offers another starting point. DUSt3R [4] and MASt3R [5] predict dense geometry from image pairs. Splatt3R [6] extends the same features with Gaussian position, shape, colour, and opacity.

This pairwise capability changes the cost of map creation, but it leaves a sequence-level problem. Every image pair defines a local coordinate frame. Repeated observations create overlapping copies of the same surface. Later loop closures move earlier camera poses. Simply concatenating the predictions therefore produces a large map whose placement and appearance can become inconsistent.

1.1 The central idea

Splatt3R-SLAM gives every local map a keyframe anchor. Its Gaussians remain in that anchor's coordinates and move when the pose graph updates the keyframe. The images used to refine the map are stored relative to the same anchors. A pose correction then moves the local map and its observation cameras together.

This shared-anchor relation is the thread through the thesis. It connects pairwise prediction to a persistent map, allows appearance refinement to run after tracking, and gives each experiment a clear role. The system follows three stages:

1. **Predict** a local Gaussian map from an image pair.
2. **Anchor** the map and its supervision to the pose graph.
3. **Refine** Gaussian appearance against tracked RGB observations.

The geometric tracker remains MASt3R-SLAM [7]. The mapping process reads its poses but does not optimise them.

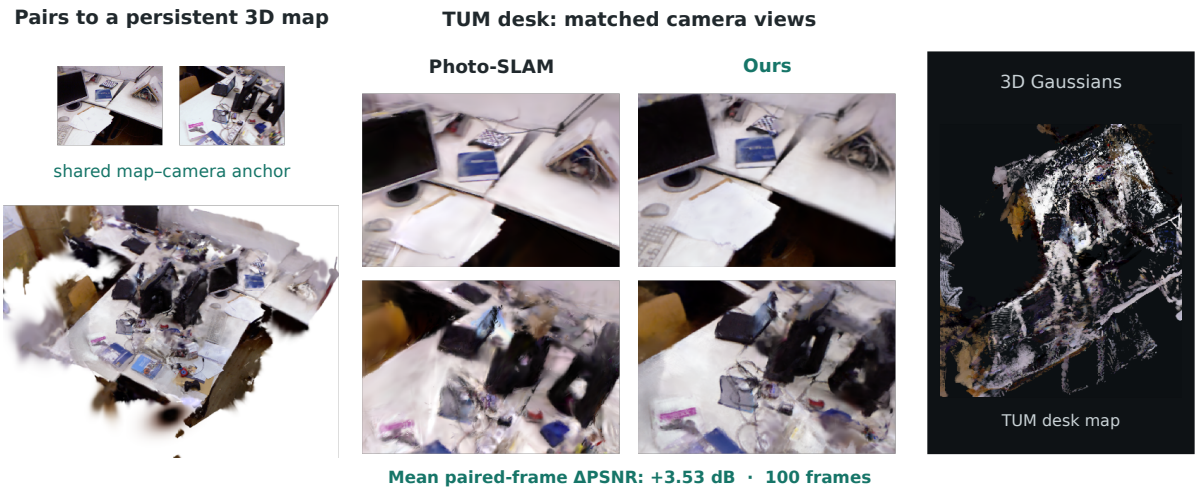


Figure 1.1. Input pairs, a pulled-back 3D view of the saved TUM desk map, and same-camera comparisons with Photo-SLAM. On all 100 TUM desk evaluation frames, the mean paired-frame PSNR gain is $+3.53$ dB; the scene-aggregated PSNR difference is $+3.28$ dB. The displayed frames are selected for high absolute reconstruction fidelity.

1.2 Research questions

The experiments address four questions:

RQ1 How can pairwise Gaussian prediction be adapted without changing the shared geometric features used for tracking?

RQ2 Can opacity attenuation at insertion reduce the effect of overlapping predictions in an assembled map?

RQ3 How much appearance improvement comes from sequence-level refinement, and can it be delivered without disrupting tracking?

RQ4 How does the complete system compare with locally executed SLAM baselines in rendering quality, trajectory accuracy, map size, and compute?

1.3 From design to evidence

Each chapter tests one part of the pipeline. Head-only adaptation changes the first stage while the encoder and decoder stay frozen. Opacity thinning changes the second stage by lowering the initial contribution of selected primitives. The refiner changes the third stage through an RGB rendering loss. Paired studies isolate these interventions before the complete system is compared with other methods.

The main external result uses the released Splatt3R head, no thinning, fixed Gaussian centres, and 300 s post-sequence refinement. It achieves state-of-the-art performance: 23.02 dB versus 19.48 dB for Photo-SLAM, with wins on all eight Replica scenes. On TUM desk, its 13.95 dB

is close to MonoGS's 13.91 dB, while MonoGS has lower LPIPS. The same evaluation also exposes the cost of dense prediction: the TUM map contains 2.397 million primitives.

1.4 Contributions

1. A pose-anchored mapping design in which local Gaussians and their supervision cameras follow the same keyframe corrections (Chapters 4 and 7).
2. A decoupled refinement process that improves appearance without updating the tracking model or camera poses, with causal, latency, and two-GPU controls (Chapters 7 and 9).
3. Paired studies of head-only adaptation and insertion-time opacity attenuation, including controls for confidence weighting and refinement (Chapters 5 and 6).
4. Common-camera reconstruction comparisons with all-eight-scene Replica coverage, four-method qualitative views, trajectory controls, and explicit map-size and resource measurements (Chapters 8 and 9).

1.5 Thesis structure

Chapters 2 and 3 introduce Gaussian rendering, pairwise prediction, and related SLAM systems. Chapter 4 presents the shared-anchor design. Chapters 5 to 7 explain the three intervention stages. Chapters 8 and 9 define and report the evaluation. Chapter 10 records tested routes that did not improve the complete system. Chapters 11 and 12 discuss the remaining problems and future work. The appendices contain configurations, reproduction steps, baseline build records, and extended tables.

Chapter 2

Background

This chapter introduces the representation, renderer, prediction models, and tracking system used by Splatt3R-SLAM. It also defines the metrics used in later experiments.

2.1 3D Gaussian splatting

2.1.1 The representation

3D Gaussian splatting [1] represents a scene as an unordered set of anisotropic Gaussians. Primitive n carries a centre $\mu_n \in \mathbb{R}^3$, a covariance $\Sigma_n \in \mathbb{R}^{3\times3}$, an opacity $\alpha_n \in (0,1)$ and a view-dependent colour encoded as spherical-harmonic coefficients. Its unnormalised spatial influence at a world point x is

$$G_n(x) \;=\; \exp\!\Big(-\tfrac{1}{2}(x-\mu_n)^\top \Sigma_n^{-1}(x-\mu_n)\Big). \tag{2.1}$$

A covariance must remain positive semi-definite under gradient descent. It is therefore factored into a rotation unit quaternion q_n and positive per-axis scale s_n:

$$\Sigma_n \;=\; R(q_n)\,\operatorname{diag}(s_n^2)\,R(q_n)^\top. \tag{2.2}$$

The scale is parameterised as $s_n = \exp(\hat{s}_n)$ elementwise, which keeps it positive without a constraint. Large changes in $\hat{s}_n$ can produce the scale excursions measured in Section 10.1.

2.1.2 Projection and rasterisation

Rendering projects each Gaussian to the image plane. Let $R_{\mathcal{C}W}$ be the rotation from world coordinates to the rendering camera and let J_n be the 2×3 Jacobian of perspective projection at the transformed centre. The EWA construction of Zwicker et al. [8] gives

$$\Sigma_n^{2D} = J_n R_{\mathcal{C}W} \Sigma_n^W R_{\mathcal{C}W}^\top J_n^\top. \tag{2.3}$$

The rasteriser sorts primitives by depth per screen tile and composites them front-to-back. Let $\mathbf{c}_\ell(u)$ be the SH-evaluated RGB colour and $b_\ell(u) \in [0,1]$ the opacity multiplied by the 2D

Gaussian footprint of the ℓ-th primitive along pixel ray u. The rendered colour is

$$\hat{I}(u) = \sum_{\ell=1}^{K} \mathbf{c}_\ell(u) b_\ell(u) \prod_{r<\ell} \big(1 - b_r(u)\big), \tag{2.4}$$

and the residual transmittance after all K primitives is

$$\mathcal{T}_K(u) = \prod_{\ell=1}^{K} \big(1 - b_\ell(u)\big). \tag{2.5}$$

2.1.3 Background and visibility

Equation (2.5) is where the background enters. A renderer composites $\hat{I}(u) \leftarrow \hat{I}(u) + \mathcal{T}_K(u)\mathbf{c}_{\text{bg}}$, so with $\mathbf{c}_{\text{bg}} = 0$ the residual transmittance contributes *zero radiance*: whatever a ray fails to hit is black. For one isolated primitive with non-negative colour, increasing opacity increases its contribution against that background. A stack is more general: changing one opacity also changes the visibility of every primitive behind it. A darker foreground primitive can hide a brighter background primitive, so increasing its opacity can lower pixel luminance. Opacity affects appearance as well as coverage, but the sign cannot be inferred from the background alone. This distinction limits the interpretation of the experiments in Chapter 6.

2.1.4 Per-scene optimisation, and what a feed-forward model skips

The reference 3DGS pipeline initialises primitives from a structure-from-motion point cloud and optimises them against posed training images under

$$\mathcal{L}_{\text{3DGS}} = (1 - \lambda_D)\, \|\hat{I} - I\|_1 + \lambda_D \big(1 - \text{SSIM}(\hat{I}, I)\big), \tag{2.6}$$

interleaved with *adaptive density control*: primitives whose view-space positional gradient exceeds a threshold are cloned (if small) or split (if large), primitives whose opacity falls below a threshold are pruned, and opacity is periodically reset so that the optimiser must re-justify every primitive it keeps. Spherical-harmonic degree is raised on a schedule, so that low-frequency appearance is fitted before view-dependent effects.

A feed-forward model emits all primitives in one pass. It has no per-scene densification, pruning schedule, or opacity reset. When predictions from many keyframes are accumulated, the map therefore retains every inserted primitive. Section 11.2 measures the resulting size. Our online refiner (Chapter 7) restores the image objective in Eq. (2.6), but not adaptive density control.

2.2 Feed-forward pointmaps and Gaussians

2.2.1 DUSt3R and MASt3R

DUSt3R [4] maps an uncalibrated image pair $I_i, I_j \in \mathbb{R}^{H\times W\times 3}$ to pointmaps $X^{(i)}_{\mathcal{C}_i}, X^{(j)}_{\mathcal{C}_i} \in \mathbb{R}^{H\times W\times 3}$ with confidences $C^{(i)}_{\mathcal{C}_i}, C^{(j)}_{\mathcal{C}_i}$. Here $X^{(b)}_{\mathcal{C}_a}$ denotes image b's pointmap expressed in camera frame $\mathcal{C}_a$. Both images pass through a shared ViT encoder; two decoders then attend to each other's tokens, so each view's prediction is conditioned on the other, and DPT heads regress the pointmaps. The output lives in the *first* camera's frame, so a single forward pass already encodes the relative pose implicitly rather than as an explicit estimate.

MASt3R [5] adds a matching head producing dense local features $D^{(i)}, D^{(j)} \in \mathbb{R}^{H\times W\times d}$, trained so that corresponding pixels are mutual nearest neighbours in feature space. We write $F_M(I_i, I_j)$ for one forward pass.

Two properties of this family govern everything downstream. The prediction is *scale-ambiguous up to a per-forward gauge* — each independent pass chooses its own scale, and nothing couples the choices — and it is *pairwise*, so no single forward pass ever observes more than two views. Section 10.5 measures scale disagreement and finds that joint multi-view context reduces it in the tested models.

2.2.2 Many-view feed-forward models

VGGT [9] generalises the formulation: all views enter one transformer with alternating frame-wise and global attention, and cameras, depth maps and pointmaps are read out jointly. The consequence that matters here is that a many-view model solves for a *shared* scale across its input window instead of drawing one per pair. Section 10.5 quantifies the difference at $6.4\times$ — and also shows that the same weights run pairwise are worse than the two-view MASt3R line, so the advantage belongs to the context, not to the architecture.

2.2.3 Splatt3R

Splatt3R [6] adds a third head, in parallel to the pointmap and matching heads and using the same DPT decoder, which emits for pixel n a rotation quaternion $q_n \in \mathbb{R}^4$, a scale $s_n \in \mathbb{R}^3_{>0}$, spherical-harmonic coefficients $S_n \in \mathbb{R}^{3\times D_{\mathrm{SH}}}$, an opacity logit $\widetilde{\alpha}_n \in \mathbb{R}$, and a mean offset $\Delta_n \in \mathbb{R}^3$. The Gaussian centre and opacity are

$$\mu_n = x_n + \Delta_n, \quad x_n \in X^{(i)}_{\mathcal{C}_i}, \qquad \alpha_n = \mathrm{sigmoid}(\widetilde{\alpha}_n). \tag{2.7}$$

D_{SH} is the number of SH basis coefficients per colour channel. Quaternions are normalised and raw scale channels are exponentiated. Offsets use a norm-dependent exponential rescaling of a signed vector, so their coordinates are not restricted to positive values. In upstream training, *only the Gaussian head is trained*: the encoder and decoder remain those of the pre-trained MASt3R

model, and all Gaussian parameters are predicted in the first image's camera frame, avoiding any ground-truth transformation at inference.

Covariance follows Eq. (2.2) and rendering follows Eq. (2.4); the Gaussians a single forward pass emits are therefore immediately renderable by a standard 3DGS rasteriser, with no optimisation step in between.

Splatt3R is supervised only by novel-view renderings, under a mask M of pixels visible in at least one context view:

$$\mathcal{L}_{\text{head}} = \mathcal{L}_{\text{MSE},M}(\hat{I}, I) + 0.25\,\mathcal{L}_{\text{LPIPS(VGG)},M}(\hat{I}, I), \tag{2.8}$$

where the subscript M denotes spatial averaging over valid pixels. Two properties of Eq. (2.8) drive our method. It is *purely photometric* — depth enters only through the mask M and through pair selection, never through a gradient on geometry — and it is *pairwise*: the loss never observes more than two context views, hence never observes a fused map.

In the released implementation the DC colour is anchored to the source image: the head predicts a residual which is added to the spherical-harmonic encoding of the input RGB value,

$$S_{\text{DC}}(u) = S_{\text{res}}(u) + \text{RGB2SH}(I_i(u)). \tag{2.9}$$

Colour is clamped during conversion; it is not sigmoid-bounded. The residual provides an appearance adjustment independently of opacity, so opacity is not the only channel capable of compensating for input photometry. Consequently, revisits can inherit exposure, white-balance, shadow, and view-dependent differences from their source frames. Exposure normalisation removes only the global component of this source-frame photometric gauge; it does not enforce equality of S_{DC} for the same world surface.

2.3 SLAM with a pointmap prior

We use T_{AB} for a transform that maps coordinates from frame B to frame A. We use the front- and back-end of MASt3R-SLAM [7]. This section summarises the tracking path whose trajectory accuracy is measured in Chapter 9.

2.3.1 Poses on the Sim(3) manifold

Because network predictions are scale-ambiguous, poses are $\text{Sim}(3)$ rather than $\text{SE}(3)$, so that a per-keyframe scale is available to absorb the ambiguity:

$$T = \begin{bmatrix} aR & t \\ 0 & 1 \end{bmatrix}, \qquad T \leftarrow \tau \oplus T \triangleq \text{Exp}(\tau) \circ T, \tag{2.10}$$

with $a > 0$ and $\tau \in \mathfrak{sim}(3)$. The scale degree of freedom absorbs per-forward gauge error. This is why Section 10.5 finds that correcting the map's scales alone makes the rendering worse.

2.3.2 Matching and the ray error

Under a generic central-camera assumption a pointmap induces its own camera model through the ray map $\psi(X) = X/\|X\|$. Two rays differ by

$$\|\psi_1 - \psi_2\|^2 = 2(1 - \cos\theta), \qquad \cos\theta = \psi_1^\top \psi_2, \tag{2.11}$$

which is *bounded*, whereas a 3D point error is not. Since pointmap predictions exhibit depth errors far more often than directional errors, tracking minimises a ray error rather than a point error. For matches $\mathcal{M}_{f,k}$ between the current frame f and keyframe k,

$$E_r = \sum_{m,n \in \mathcal{M}_{f,k}} \left\| \frac{\psi(\widetilde{X}^{(k)}_{\mathcal{C}_k,n}) - \psi(T_{\mathcal{C}_k\mathcal{C}_f} X^{(f)}_{\mathcal{C}_f,m})}{w(\omega_{m,n}, \sigma_r^2)} \right\|_\rho, \tag{2.12}$$

with $\|\cdot\|_\rho$ a Huber norm, $\omega_{m,n}$ the match confidence, and w its confidence-dependent noise scale; invalid matches receive infinite scale. A small term on the difference of distances from the camera centre keeps the pure-rotation case non-degenerate. The system is solved by Gauss–Newton in an iteratively-reweighted least-squares loop:

$$\big(J^\top W J\big)\tau = -J^\top W r, \qquad T_{\mathcal{C}_k\mathcal{C}_f} \leftarrow \tau \oplus T_{\mathcal{C}_k\mathcal{C}_f}. \tag{2.13}$$

Both the iterative projection and the feature-space refinement of matches are implemented as per-pixel CUDA kernels, which is what keeps tracking in the low-millisecond range.

2.3.3 Keyframing and canonical pointmap fusion

A new keyframe is created when the number of valid matches, or of unique keyframe pixels among them, falls below a threshold. Each keyframe maintains a canonical pointmap updated by a confidence-weighted running average,

$$\begin{aligned} \widetilde{X}^{(k)}_{\mathcal{C}_k} &\leftarrow \frac{\widetilde{C}^{(k)}_{\mathcal{C}_k} \widetilde{X}^{(k)}_{\mathcal{C}_k} + C^{(k)}_{\mathcal{C}_f} T_{\mathcal{C}_k\mathcal{C}_f} X^{(k)}_{\mathcal{C}_f}}{\widetilde{C}^{(k)}_{\mathcal{C}_k} + C^{(k)}_{\mathcal{C}_f}}, \\ \widetilde{C}^{(k)}_{\mathcal{C}_k} &\leftarrow \widetilde{C}^{(k)}_{\mathcal{C}_k} + C^{(k)}_{\mathcal{C}_f}. \end{aligned} \tag{2.14}$$

Filtering merges information from many viewpoints into geometry that no single small-baseline prediction could support. Note what Eq. (2.14) does *not* do: it averages within a keyframe, never across keyframes, so it cannot remove a disagreement between two keyframes' scales. That disagreement is the subject of Section 10.5.

2.3.4 Loop closure and the back-end

Loop-closure candidates are retrieved with an incremental adaptation of the Aggregated Selective Match Kernel (ASMK) [10] over encoded features, as used by MASt3R-SfM. Each new keyframe queries the database for top-K candidates; those above a score threshold are passed to the decoder, and bidirectional edges are added when the resulting match count clears a second threshold. The keyframe's features then enter the inverted file index. Section 10.3 reports a closed attempt to refit these assets on our own encoder's features.

Global consistency is obtained by minimising the same ray error over all edges $\mathcal{E}$ of the keyframe graph,

$$E_g = \sum_{(i,j)\in\mathcal{E}} \sum_{m,n\in\mathcal{M}_{i,j}} \left\| \frac{\psi(\widetilde{X}^{(i)}_{\mathcal{C}_i,m}) - \psi(T_{\mathcal{C}_i\mathcal{C}_j}\widetilde{X}^{(j)}_{\mathcal{C}_j,n})}{w(\omega_{m,n},\sigma_r^2)} \right\|_\rho, \quad T_{\mathcal{C}_i\mathcal{C}_j} = T_{W\mathcal{C}_i}^{-1} T_{W\mathcal{C}_j}, \tag{2.15}$$

solved by second-order Gauss–Newton with sparse Cholesky over the $7N \times 7N$ Hessian. The first $\mathrm{Sim}(3)$ pose is fixed to remove gauge freedom. Splatt3R-SLAM does not change this solver; Section 9.6 compares its trajectory with the inherited control.

2.4 Metrics

2.4.1 Image metrics

PSNR is a log-scaled mean-squared error, $\mathrm{PSNR} = 10\log_{10}(1/\mathrm{MSE})$ for images in $[0,1]$. It is sensitive to global photometric offsets — a uniform brightness shift costs PSNR without changing structure — which is why the protocol of Chapter 8 normalises exposure identically for every system before scoring. SSIM [11] compares local luminance, contrast and structure statistics. LPIPS [12] compares deep features of a pre-trained network. Reconstruction evaluation uses LPIPS-AlexNet; head adaptation retains the upstream VGG-based LPIPS loss.

We report PSNR and LPIPS together because they differ in Table 9.5: PSNR improves on $8/8$ scenes and LPIPS on $5/8$. LPIPS is not part of the refiner loss (Eq. (7.1)).

2.4.2 Trajectory metrics

Absolute Trajectory Error is translation RMSE after alignment to ground truth. Monocular scale ambiguity requires a $\mathrm{Sim}(3)$ alignment, solved by Umeyama's closed-form method [13]. All five systems in Table 9.8 use `evo_ape tum -as`.

Rendering also requires transporting ground-truth cameras into each map's similarity gauge; Chapter 8 specifies this separate use of alignment.

Chapter 3

Related Work

3.1 Feed-forward 3D reconstruction

DUSt3R [4] recasts two-view stereo as direct regression of aligned pointmaps, removing explicit correspondence search; MASt3R [5] adds a matching head and metric scale, and VGGT [9] generalises the formulation to many views. Splatt3R [6] builds on MASt3R by freezing the encoder and decoder and training only a Gaussian head that emits, per pixel, a covariance, spherical-harmonic colour and opacity (Eq. (2.7)–Eq. (2.8)). Its head is trained against frozen features, motivating our restricted adaptation study (Section 5.1). The pairwise photometric objective does not directly observe the interactions of hundreds of accumulated keyframes in a SLAM map.

3.2 Neural and Gaussian map representations

Dense SLAM has passed through three map representations in quick succession. Volumetric TSDF fusion gives geometry but no photorealistic appearance; neural implicit fields, following NeRF [14], give appearance with potentially substantial rendering and incremental optimisation cost. 3D Gaussian splatting [1] offers two useful properties that belong to the representation rather than to any particular system: rasterisation is explicit and fast, and the primitives are local, so an update in one part of the scene does not require re-evaluating a global network. Locality also makes anchor-carried composition (Section 4.2) straightforward: pose corrections can be applied to local primitives without re-optimising their attributes.

3.3 Gaussian-splatting SLAM

MonoGS [2] optimises Gaussians and camera poses jointly by gradient descent from monocular video. Photo-SLAM [3] couples ORB-SLAM3 [15] tracking with a Gaussian mapping thread. Splat-SLAM [16] deforms an RGB-only Gaussian map after global pose and depth updates. SEGS-SLAM [17] builds structured anchors from a tracked point cloud and conditions appearance on camera motion. SplaTAM [18] and Gaussian-SLAM [19] take related routes with RGB-D input. Unblur-SLAM [20] combines feed-forward deblurring with rendering-based test-time refinement for blurred observations. Its separation of learned initialisation and scene-

specific refinement is related to our design, but we address pairwise Gaussian accumulation under a changing pose graph and introduce no blur model. The optimisation-based systems obtain map primitives by per-scene optimisation, and their map quality is therefore a function of the optimisation budget they are given — a dependency we make explicit when comparing (Chapter 9). Our system differs in that primitives arrive *predicted*: the optimiser refines an initialisation that is already approximately correct, rather than creating structure.

3.4 Feed-forward priors inside SLAM

DROID-SLAM [21] established learned dense optimisation for tracking. MASt3R-SLAM [7] uses the MASt3R prior for real-time dense tracking and pointmap fusion, and is the system we build on; we inherit its tracking front-end unmodified, which is why our trajectory accuracy matches it and why we are explicit that trajectory accuracy is *not* a contribution of this work. VGGT-SLAM [22] aligns VGGT submaps on the $SL(4)$ manifold; it produces poses and a point cloud but no renderable map, so in our comparison it serves as a tracking control rather than a rendering baseline.

3.5 Position of this work

Splatt3R-SLAM combines the fast initialisation of feed-forward prediction with sequence-level refinement. Unlike pairwise reconstruction, it must maintain local predictions under pose-graph updates. Unlike optimisation-first Gaussian SLAM, it begins with a dense primitive at almost every valid pixel. The shared-anchor design addresses the first difference. The adaptation, thinning, and refinement studies address the second.

3.6 Parameter-efficient adaptation

LoRA [23] adapts a backbone through low-rank updates. Splatt3R instead trains its Gaussian head against frozen MASt3R features. We compare these choices in Chapter 5. The tested encoder route loses 49% reconstruction quality, while head-only training improves several dataset families.

3.7 Evaluation practice

PSNR/LPIPS [12] and similarity-aligned ATE [13, 24] depend on scoring views, camera poses, resolution, preprocessing, and optimisation budget. Chapter 8 defines one common renderer and camera set for the local comparison.

Chapter 4

System Design

4.1 Overview

Splatt3R-SLAM uses four processes when refinement and the GUI are enabled (Fig. 4.1). The *tracker* consumes frames, runs the feed-forward backbone, solves Eq. (2.13) for the relative pose against the current keyframe, and decides when to create a new keyframe. The *back-end* maintains the keyframe graph, queries retrieval for loop candidates and solves Eq. (2.15). The *refiner* (Chapter 7) is optional and photometrically improves the assembled map while the other two run. The *visualiser* renders the current map and subscribes to refined snapshots. Both headless experiments and GUI-enabled dual-GPU delivery measurements are reported, with their regimes identified. Figure 4.1 unifies the method and execution paths.

The refiner runs in its own process. This allows it to use a different GPU, which Chapter 7 shows reduces contention in the measured TUM desk configuration. It also separates photometric optimisation from geometric tracking: the refiner reads the pose state but does not optimise or write tracking poses. Resource contention can still affect scheduling and end-to-end throughput.

SharedKeyframes stores images, pointmaps, confidences, poses and Gaussian payloads in CUDA tensors exposed through inter-process communication. The refiner transfers new Gaussian payloads to its device and reads current anchor poses. Separately, tracked RGB supervision and anchor-relative camera transforms live in the CPU SupervisionFrames pool. Complete refined maps are published through a CPU snapshot buffer. These are distinct storage paths.

4.2 Anchor-carried map composition

We replace the pointmap prediction with Splatt3R, so each keyframe pair yields a per-pixel Gaussian in one forward pass. Keyframe Gaussians are stored in their anchor keyframe's local frame and composed into the world through that keyframe's *current* pose,

$$\mu_n^W = a_k R_k \mu_n^{\mathcal{C}_k} + t_k, \qquad \Sigma_n^W = a_k^2 R_k \Sigma_n^{\mathcal{C}_k} R_k^\top, \tag{4.1}$$

where $T_{W\mathcal{C}_k} = (a_k R_k, t_k)$ is the camera-to-world $\mathrm{Sim}(3)$ pose of keyframe k. Because the composition reads the pose at render time, a loop closure that updates $T_{W\mathcal{C}_k}$ moves the map without rewriting local parameters.

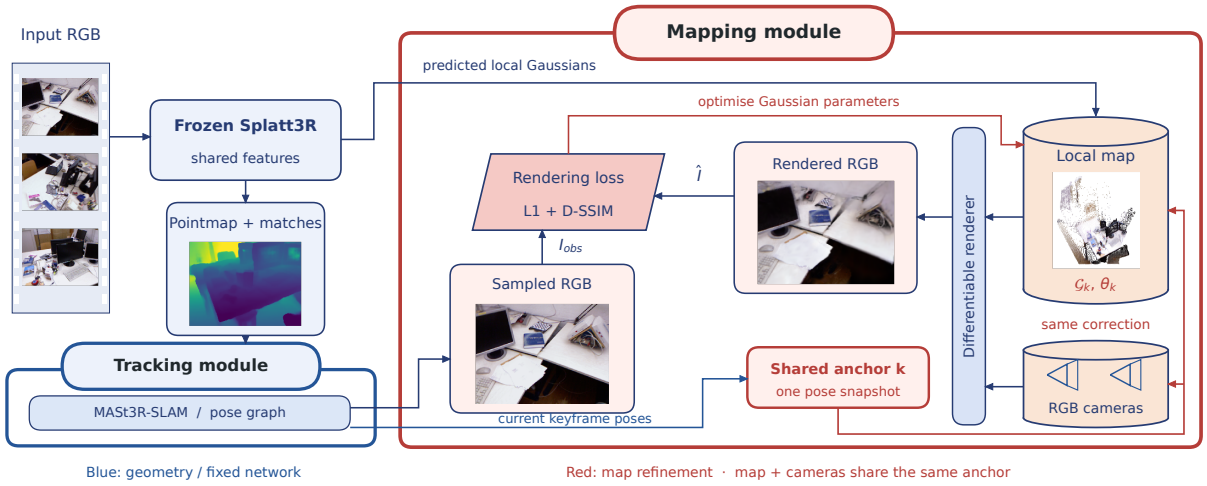


Figure 4.1. Frozen prediction, geometric tracking (blue) and Gaussian mapping (red). The two cylinders hold a local Gaussian map and anchor-relative supervision cameras. Both use one current pose snapshot and move with the same keyframe correction. Photometric gradients update only Gaussian parameters; tracking poses and the frozen backbone are excluded.

Supervision uses the same anchor. If tracked frame f stores a relative pose $T_{\mathcal{C}_k\mathcal{C}_f}$, then

$$T_{W\mathcal{C}_f} = T_{W\mathcal{C}_k} T_{\mathcal{C}_k\mathcal{C}_f}. \tag{4.2}$$

For a local map and camera sharing anchor k, their relative transform is

$$T_{W\mathcal{C}_f}^{-1} T_{W\mathcal{C}_k} = (T_{W\mathcal{C}_k} T_{\mathcal{C}_k\mathcal{C}_f})^{-1} T_{W\mathcal{C}_k} = T_{\mathcal{C}_k\mathcal{C}_f}^{-1}. \tag{4.3}$$

The anchor cancels, so any invertible update of $T_{W\mathcal{C}_k}$ preserves this relation. The identity applies within one anchor; neighbouring local maps can still disagree.

The implementation stores means, scales, rotations, SH coefficients, opacity, confidence, and validity in a shared keyframe buffer. Tracker updates may rewrite pointmap state without clearing the Gaussian record unless a new Gaussian prediction is present. This invariant prevents a geometric update from silently erasing an already renderable keyframe. Filtering and the global Gaussian limit are applied before rasterisation; the limit is therefore a rendering budget implemented through spatial stride, rather than FIFO deletion of individual keyframes.

4.3 The injection pipeline

Between "the network emitted a Gaussian" and "the Gaussian is in the map" sit several filters, and their order matters because two of them interact with the contribution of Chapter 6.

1. **Validity.** Pixels without a valid pointmap prediction emit no Gaussian.
2. **Confidence gate.** Gaussians below `--refiner-min-confidence` are dropped outright. This is a *deletion* operation and is deliberately kept distinct from thinning, which is

not.

3. **Scale clamp.** An optional cap on injected scale, applied at refinement time and independent of the bake-time cap, bounds the damage a single mispredicted $\hat{s}$ can do to a render.

4. **Band-limiting.** Injected Gaussians are optionally band-limited against the measured lattice pitch of the prediction grid (`-refiner-aa-sigma`), which removes the dot/moiré pattern that a per-pixel Gaussian field otherwise produces when viewed off-axis.

5. **Opacity thinning.** Equation (6.1), the subject of Chapter 6.

The global Gaussian limit is applied *before* rasterisation and is implemented as a spatial stride over the map, not as FIFO deletion of keyframes. The distinction is not cosmetic: a FIFO policy would make the displayed map a function of how long the sequence has been running, whereas a stride makes it a function of the display budget alone, and leaves the evaluation artefact — the full map, written to disk — untouched by whatever the viewer could afford to draw.

4.4 Rendering, display and the evaluation artefact

The refiner publishes a versioned snapshot in CPU memory rather than exposing mutable GPU state. The visualiser uploads only complete versions and retains the previous valid version if it collides with a write. If the display buffer is smaller than the refined map, publication subsamples — but only after the full map has been saved. This ordering means a failure to publish a large map for display cannot discard a valid refinement result, which is a failure mode we hit before separating the two paths.

On termination the system writes the refined map as a standard 3DGS `.ply`, so that it can be scored, inspected or rendered by any tool in that ecosystem rather than only by ours. The external rendering comparisons in Chapter 9 use these artefacts and the full-SH evaluation script. Earlier component ablations use the recorded map-scoring harness for their respective paired runs. Neither uses viewer screenshots as quantitative data.

4.5 Changes to MASt3R-SLAM

We replace the pointmap prediction with Splatt3R's joint pointmap-and-Gaussian prediction; we add the Gaussian record to the shared keyframe buffer; we add the injection pipeline of Section 4.3; and we add the refiner process of Chapter 7. We change nothing in matching (Eq. (2.12)), nothing in keyframe selection, nothing in the retrieval path, and nothing in the back-end solver (Eq. (2.15)).

The one place where this could leak is the pointmap itself, since Splatt3R and MASt3R share an encoder and decoder but the fine-tuned head is ours. Head-only training (Chapter 5) keeps the encoder and decoder bit-frozen, so the pointmap the tracker consumes is bit-identical to what

MASt3R would have produced. This is why Section 9.6 reports parity rather than improvement; it serves as a control for the inherited tracking path.

Chapter 5

Adapting the Feed-Forward Head

5.1 Choosing the trainable module

Splatt3R trains its Gaussian head against frozen encoder and decoder features. We compare three adaptation choices:

- **Encoder LoRA** [23] (r=8, α=16 on `qkv`/`proj`/`fc1`/`fc2`): -49% reconstruction quality against the released checkpoint, with predicted scales exploding by $1821\times$. The failure is immediate, not a late-training instability.
- **Decoder-only LoRA**: $+0.37$ dB at epoch 1, then collapse to -1.61 dB by epoch 5 with a $12\times$ climb in the 99th-percentile scale.
- **Head-only** (encoder and decoder bit-frozen, only `gaussian_dpt` trainable): improves, and is what we keep.

The collapse coincides with the exponential Gaussian-scale parameter $s_n = \exp(\hat{s}_n)$. A small feature shift can produce a large scale change, while different scale–opacity combinations can fit the same supervision views. We therefore retain head-only training for the evaluated system. The shared encoder, decoder, and geometric heads remain frozen; the Gaussian DPT head uses the upstream objective and Adam [25].

5.2 Training setup

Adaptation is performed per dataset family. The families differ in sensor, exposure, motion, and depth availability. Each receives its own head checkpoint, loaded at start-up.

The objective is the upstream one of Eq. (2.8), unchanged, so that the comparison against the released checkpoint isolates the data and the choice of what to train, not the loss. Only `gaussian_dpt` receives gradients; the encoder and decoder are frozen, and this is verified by comparing parameter hashes before and after training rather than by trusting a `requires_grad` flag.

The TUM, EuRoC and 7-Scenes heads in the reported paired adaptation study were trained for 40 epochs. Their batch sizes are 2, 4 and 8, with learning rates 10^{-5}, 1.41×10^{-5} and 2×10^{-5}, respectively. A 15% validation split is drawn within the available family data. These

experiments measure in-family adaptation; they do not establish transfer to an unseen dataset family or a sequence-disjoint benchmark.

Two of the five families (EuRoC and ETH3D) ship no usable depth for the training pairs. Rather than exclude them, pairs are supervised with self-predicted pseudo-depth from the frozen backbone. This is a weaker supervision signal, and the results in Chapter 9 should be read with that in mind: EuRoC is the family where the refiner contributes least ($+1.00$ dB) and ETH3D the family where it contributes most ($+8.69$ dB), and we do not have an explanation for that spread (Section 10.4).

5.3 Photometric augmentation and the opacity channel

Section 6.3 examines opacity's photometric role. Inputs darkened independently of target views provide a controlled way to test whether opacity statistics respond to appearance perturbations. The SH residual can also change colour, so this experiment does not isolate opacity as the only compensating channel.

Four augmentation regimes were trained for 40 epochs each on the same family and probed with an identical five-pair protocol. The table reports the 90th-percentile predicted scale (a proxy for how far the head has moved from the released checkpoint's geometry), the median predicted opacity, and the fraction of Gaussians with opacity above 0.9 — i.e. how saturated the opacity channel is:

Head	scale p90	inflation	median α	$\alpha > 0.9$
base (released)	9.78	—	1.000	100.0%
content noise + blur	26.18	$+167.8\%$	1.000	100.0%
global photometry	17.37	$+77.6\%$	0.991	83.1%
spatial photometry	14.82	$+51.5\%$	0.876	42.9%
content + spatial photometry	10.97	$+12.2\%$	0.933	62.7%

Three things are visible. The released checkpoint's opacity is *completely saturated* — median 1.000, every Gaussian above 0.9 — which provides little initial differentiation between primitives and motivates testing an external attenuation prior. Content-only augmentation does not move the channel at all while inflating scale by 168%. Global photometry modestly reduces saturation; spatial photometry reduces it much more strongly, from 100% to 42.9%. These are observed parameter responses, rather than a complete account of what the head learned.

The combined regime was tested against an explicit prediction: if the two perturbations engage independently, saturation should fall to 30–50% *and* scale inflation should exceed 10%. Scale inflation cleared the bar at $+12.2\%$, barely; saturation landed at 62.7%, *above* the predicted range and above what spatial photometry achieved on its own. Both numbers moved in the direction of mutual suppression rather than independent engagement. The prediction is therefore not supported.

Across four scenes the spatial-photometry head gives a mean LPIPS improvement of -14.55%, with a per-scene spread of -6.6% to -19.2%. The gain is scene-dependent.

Chapter 6

Injection-Time Thinning and the Opacity Mechanism

6.1 The crowding problem

A two-view prediction covers each visible surface once. A SLAM map covers it once *per keyframe that saw it*, and on a typical indoor sequence that is tens of times. Nothing in the pipeline reconciles these copies: each is predicted independently, placed by its own keyframe's pose, and injected with whatever opacity its forward pass produced.

Under Eq. (2.4) the consequence is specific rather than generic. If the copies agreed in colour and depth, stacking would still change effective opacity: K copies with opacity α have combined opacity $1-(1-\alpha)^K$. Inconsistent copies additionally differ in depth by the per-forward scale gauge (Section 10.5 measures this at $\sim 5\%$), in colour by the source-frame photometric anchoring of Eq. (2.9), and in extent by the head's own scale prediction. A ray therefore accumulates opacity from several slightly-displaced, slightly-different surfaces before its transmittance runs out, and what should be one sharp surface composites into haze.

The released checkpoint makes this worse in a way that is visible in its statistics rather than in its images: its predicted opacity is *saturated*, with median 1.000 and every Gaussian above 0.9 (Section 5.3). A saturated opacity channel cannot express "this copy should contribute less". The lever below supplies that expression from outside the network.

6.2 Insertion-time opacity attenuation

A SLAM map covers each surface with predictions from many keyframes. Under Eq. (2.4) with a black background, a surface covered by many over-solid Gaussians composites to haze rather than to a sharper surface. We therefore fade opacity at the moment a Gaussian enters the map, in proportion to how little the pointmap confidence vouches for it.

Confidences are not calibrated across keyframes, so we rank-normalise *within* each keyframe. For a keyframe with N Gaussians and confidence values $\{\gamma_n\}$, let $r_n \in \{0,\dots,N-1\}$ be the ascending rank of γ_n and define $\hat{\gamma}_n = r_n / \max(N-1,1) \in [0,1]$. The injected opacity is

$$\alpha'_n \;=\; \alpha_n \cdot \mathrm{clip}\Big(1-2\lambda(1-\hat{\gamma}_n),\; 0.1,\; 1\Big), \tag{6.1}$$

with dose λ=0.45 in the thinning experiments. External comparisons disable thinning with $\lambda = 0$. The lower clip at 0.1 retains every primitive and its refinement gradient.

6.2.1 Relation to pairwise training

An opacity penalty added to Eq. (2.8) would act during pair training. Crowding appears later, after SLAM accumulates keyframes over a surface, and Eq. (2.8) never sees more than two context views. Consistent with this, we measured that the refiner un-saturates opacity on its own even with the released head (median $1.0 \rightarrow 0.26$). Attenuation therefore starts the map closer to an opacity state that the optimiser can reach. A dose–response check at two thinning depths (opacity knees 0.9 and 0.6) gave -21.0% and -15.8% LPIPS. Benefits persist for these heads, supporting complementarity in this experiment. A different training objective or sequence-level training procedure might still learn an equivalent adjustment.

6.3 Opacity, colour, and visibility

The DC colour is an SH residual added to the encoding of input RGB, with clamping at conversion (Eq. (2.9)). Both the residual and opacity can respond to photometric perturbations. Opacity additionally controls which surfaces remain visible along a ray.

For an isolated primitive over black, greater opacity increases its contribution. In a multi-colour stack it may instead obscure brighter content. The background therefore does not imply a monotone relation between opacity and brightness. We test their correlation without assigning a unique cause.

We registered a sign-specific prediction. Let ℓ be input luminance and ρ the Spearman correlation between α and ℓ over predicted Gaussians. A head trained with independently darkened inputs (synthetic vignetting and white-balance jitter) was predicted to have $\rho < 0$, whereas a real-sensor head without synthetic darkening was predicted to have $\rho > 0$. Section 9.3 reports both signs in the five tested heads: two Replica augmentation heads and three real-data heads. These are five models, not five independent dataset families. Correlations are consistent with opacity responding to photometry, but do not identify a unique causal mechanism.

6.4 Controls

6.4.1 Confidence weighting

Equation (6.1) fades in proportion to rank-normalised confidence, and the name “confidence-weighted thinning” invites the reading that the confidence signal is what makes it work. We measured the alternative — the same total dose applied *uniformly*, with no confidence term at

all — and against confidence weighting it is statistically a coin flip: $n{=}12$ online cells, median difference -0.06%.

Reducing injected opacity is the robust effect in this study. The confidence-weighted form concentrates reduction where the pointmap is least sure, but the uniform control performs similarly.

6.4.2 Shape-weighted attenuation

A second opacity lever reduces each Gaussian's opacity in proportion to how elongated it is relative to its local surface sampling. It targets trailing primitives near depth discontinuities. In measurement it fades 64–98% of all Gaussians by 39–65% of their opacity. Its effect is therefore broad rather than sparse.

LPIPS improves on the three tested sequences by -12.7%, -6.6%, and -1.3%, with a PSNR change of -0.04 to -0.22 dB. The LPIPS gain is largest in scenes with more streaked geometry.

Chapter 7

Online Map Refinement

7.1 Design

Prediction supplies an initialisation. A separate process photometrically refines the assembled map *while SLAM runs*; together with the GUI, the complete system has four processes (Fig. 4.1).

7.1.1 Parameterisation

Gaussians are optimised in their anchor keyframe's local frame in pre-activation form — $\log s_n$, unnormalised q_n, $\operatorname{logit} \alpha_n$, and the SH DC term — so that scales stay positive and opacity stays in $(0, 1)$ without constraints, and composed to the world by Eq. (4.1) at every step. Per-group Adam rates follow the 3DGS reference, with the positional rate scaled by scene extent: $\eta_\mu = 1.6 \times 10^{-4} \cdot \text{extent}$, $\eta_{f_{dc}} = 2.5 \times 10^{-3}$, $\eta_\alpha = 5 \times 10^{-2}$, $\eta_s = 5 \times 10^{-3}$, $\eta_q = 1 \times 10^{-3}$. Higher-order SH coefficients, when present, remain fixed.

7.1.2 Objective

Supervision views are tracked frames, so the refiner optimises

$$\mathcal{L}_{\text{ref}} = (1 - \lambda_D)\, \|\hat{I} - I\|_1 + \lambda_D \left(1 - \text{SSIM}(\hat{I}, I)\right), \qquad \lambda_D = 0.2, \tag{7.1}$$

i.e. the 3DGS objective rather than Eq. (2.8): LPIPS is a reporting metric here, not a training signal, so that our evaluation metric is not also our loss.

7.1.3 Anchor-carried supervision

Each supervision frame is stored as an image together with $(\text{anchor index}, T_{\mathcal{C}_k \mathcal{C}_f})$ rather than a world pose. Its world pose is recomposed through the anchor's *current* pose at sample time, so supervision follows loop closures exactly as the map does under Eqs. (4.1) and (4.2). Frames are held in CPU shared memory, which allows the refiner to run on a second GPU. The refiner consequently never becomes a second pose-graph writer: a loop closure changes the anchor pose once, and both map placement and supervision views follow that correction.

7.1.4 Sampling

Supervision is drawn from a reservoir of 200 frames spanning the whole sequence plus a ring of the 64 most recent, with a recent fraction of 0.3. We measured uniform-over-history to dominate a recency-weighted schedule (held-out early/late split: $14.40/14.09/14.72$ uniform versus $14.06/13.79/14.35$ recent-only), because causal arrival already biases the pool toward recent views.

7.1.5 Scheduling

Sharing one GPU, the refiner holds a duty cycle d by sleeping $\mathrm{EMA}[\Delta t]\,(1-d)/d$ after each step, so optimisation steps are dropped but never frames. Refiner load on the *same* card doubles tracker latency ($101 \to 206$ ms p50); on a second card it is noise (103 ms), which is the configuration we deploy. After the sequence ends an optional polish phase continues with the final poses.

7.1.6 Centre updates

The external-comparison configuration holds μ fixed. Earlier fixed-map experiments also favoured this choice at approximately 300 and 3000 steps. The broader causal delivery study did not show the same preference. External comparisons and causal studies therefore report their centre settings separately from the command-line defaults in Chapter A.

When centres are fixed, the positional learning rate above is inactive. When enabled, centre updates can trade geometric consistency against fit to supervision cameras. Section 9.8 separates the tested causal configuration from the fixed-centre, post-sequence comparison.

The refiner publishes a versioned CPU snapshot rather than mutable GPU state. The visualiser uploads only complete versions and retains the previous valid version on a read/write collision. If the display buffer is smaller than the refined map, publication subsamples after the full map has been saved. This separates bounded display memory from the evaluation artifact and prevents a large-map publication failure from discarding a valid refinement result.

7.2 The polish phase

When the sequence ends, the trajectory stops changing but the map is still under-optimised: every step taken during the sequence was taken against poses that were still moving. A *polish* phase continues refinement with the final poses, and it is the single largest online win we measured: $+2.21$ dB PSNR and -16.7% LPIPS over seven TUM `fr1` sequences.

The required duration varies. Sequences need between 1445 and 3821 steps to converge — a $2.6\times$ spread — and no static quantity we tried predicts which sequence needs which: not length, not keyframe count, not final Gaussian count, not the loss at the start of the phase. A fixed step budget therefore either wastes time on the easy sequences or truncates the hard ones.

The implementation offers a convergence criterion (`-refiner-polish-tol` with a patience of two logging windows), disabled by default. Its stopping rule and fixed-duration experiments should be reported separately.

Table 9.7 uses a fixed 300 s budget for our external comparison, rather than convergence-based stopping. Its PSNR is close to MonoGS after 339 s colour refinement, while MonoGS has lower LPIPS. A separate historical 1200 s run reached 14.61 dB and 0.379 LPIPS under the older evaluation configuration. That experiment demonstrates further optimisation headroom; it is not mixed into the fresh full-SH comparison or presented as a gain at matched total compute.

7.3 Density control

The refiner reinstates the 3DGS objective (Eq. (2.6)) but not the adaptive density control that accompanies it in the reference implementation. This choice has a measurable map-size cost.

Cloning and splitting are omitted because the map already has an excess of primitives rather than a shortage: the failure mode this thesis addresses is crowding, and a densification policy tuned for a sparse SfM initialisation would be solving the opposite problem. Pruning and periodic opacity reset are also omitted. Pruning changes the primitive count and requires corresponding updates to optimiser and snapshot state. Opacity reset changes the optimisation schedule.

The cost is visible in Section 11.2: without pruning, nothing ever removes a primitive that the data does not support, and the map size is whatever the injection pipeline produced. A voxel-based deduplication pass was implemented as a partial substitute and measured as a negative: it removes 27.6% of the map for -0.53 dB. We report it as a map-size control that costs quality when it fires without a recovery budget, and it is off by default. Refining directly under a primitive budget — rather than pruning after the fact — is the future work named in Chapter 12.

7.4 Tracking preservation

The refiner reads and composes poses through Eqs. (4.1) and (4.2), without optimising them. The recorded ablations preserve ATE (0.016975 m, rounding to the 0.0170 m baseline) while changing map quality by several dB. This supports the intended separation of mapping and tracking. It does not eliminate resource contention or guarantee identical scheduling in every multi-process run.

Chapter 8

Evaluation Protocol

All comparison maps and trajectories come from local runs on a host with two RTX A6000 GPUs. This chapter defines the common full-SH renderer, cameras, alignment, and aggregation.

System	Type	Rend.	Role here
Ours	mono 3DGS	✓	—
Photo-SLAM	mono 3DGS	✓	rendering
MonoGS	mono 3DGS	✓	rendering (TUM)
MASt3R-SLAM	mono dense	—	tracking control
VGGT-SLAM	pose+cloud	—	tracking control

Table 8.1. Systems compared: Photo-SLAM [3], MonoGS [2], MASt3R-SLAM [7], VGGT-SLAM [22]. The latter two supply trajectory comparisons only; MASt3R-SLAM is the inherited tracking control.

8.1 Common rendering conditions

For each scene, 100 frames are selected outside the union of participating systems' keyframes. Each exported map is rendered at these same cameras by one implementation, with identical exposure normalisation and metric code. The resolution is 512×288 on Replica and 512×384 on TUM. All exported spherical-harmonic bands are retained: degree three for Photo-SLAM, degree zero for the evaluated MonoGS and our maps.

Scene PSNR is computed as $-10 \log_{10}$ of the mean frame MSE, rather than as the arithmetic mean of per-frame PSNR. LPIPS-AlexNet [12] is averaged across frames. Dataset means weight each scene equally. Scene PSNR and the median paired frame difference can therefore rank methods differently.

8.1.1 Evaluation scope

We call these *non-keyframe reconstruction* scores. Tracked non-keyframes may enter the refiner's supervision pool. Their exclusion from the keyframe set therefore does not guarantee they were unseen during optimisation. Ground-truth cameras reduce dependence on each system's own evaluation-pose estimates, but expose tracking and alignment errors as well as map errors. Common rendering also cannot remove differences in input preprocessing, undistortion or optimisation history.

A strict novel-view experiment would exclude scoring views from every optimisation and model-selection path.

8.1.2 Datasets and configurations

Rendering covers all eight Replica [26] scenes and TUM RGB-D [27] `freiburg1_desk`, with monocular input for all compared systems. Trajectory accuracy covers all nine TUM freiburg1 sequences. Adaptation and refinement studies additionally use EuRoC [28], ETH3D [29] and 7-Scenes [30].

The external rendering configuration uses the released Splatt3R head, fixed Gaussian centres, insertion thinning disabled and 300 s post-sequence refinement. Photo-SLAM runs to its own shutdown schedule, so Replica optimisation budgets are not matched. MonoGS uses a 339 s colour-refinement phase on TUM desk. That phase is close in duration to our polish, but is not a match of total pipeline compute.

Family-head adaptation and thinning use separate paired runs. Maps delivered at sequence end are reported separately from post-sequence polish.

8.1.3 Alignment

Each monocular map has its own similarity gauge. We fit a map-to-ground-truth alignment (s_A, R_A, t_A) to corresponding estimated and ground-truth camera centres using Umeyama's method [13], such that $p_{\text{gt}} = s_A R_A p_{\text{map}} + t_A$. A ground-truth camera-to-world pose is transported into map coordinates by

$$R_{\text{map}} = R_A^\top R_{\text{gt}}, \qquad t_{\text{map}} = R_A^\top (t_{\text{gt}} - t_A)/s_A. \tag{8.1}$$

The inverse scale acts on translation, not on the rotation block. Putting it inside the rotation would produce a non-orthonormal camera orientation. ATE is translation RMSE after similarity alignment, evaluated with `evo_ape tum -as` [24].

MonoGS records estimated and ground-truth keyframe poses together. Its TUM keyframes are recovered from those saved GT poses and checked against GT orientation. This avoids confusing indices in its RGB-D-associated stream with indices in the raw RGB sequence. The updated comparison uses this association throughout selection, alignment and rendering.

8.2 Self-consistency and provenance

Before interpreting a cross-system margin, the exported map is rendered at the poses from which it was built. A poor result here requires checking camera conventions, image association and map loading before interpreting it as a property of the method. In an earlier MonoGS diagnostic, interpreting camera-to-world poses as world-to-camera changed the measured score from 18.71

to 4.36 dB. Those are diagnostic values from that experiment, not the updated common-camera scores in Table 9.7.

The current evidence records the input PLY and trajectory hashes and sizes, alignment transforms, all scoring frame indices, per-frame metrics and representative-view selection. It also records DC-only scores for comparison with the older renderer, which discarded higher SH bands. The updated Photo-SLAM Replica mean is 19.483 dB with full SH, compared with 19.581 dB in that DC-only evaluation. The external tables use full SH consistently; historical ablations retain their original configurations.

Qualitative views are selected at the two middle ranks of the per-frame Ours-minus-baseline PSNR difference: MonoGS on TUM and Photo-SLAM on Replica. All methods receive the same crop coordinates and image scale. This makes selection repeatable without selecting the most favourable tail.

8.3 Pose-source sensitivity

Earlier diagnostics rendered an unchanged MonoGS map from its estimated poses and from aligned ground-truth poses, obtaining 18.71 and 9.87 dB, respectively. A separate Photo-SLAM diagnostic measured a smaller difference of approximately 2.6 dB. These experiments demonstrate that map quality is coupled to the camera poses used to evaluate it. They also include alignment error and the degree to which map parameters have adapted to their training cameras.

An earlier local-to-published comparison changed views, resolution, masking, undistortion, and optimisation budget at once. It cannot isolate a fixed protocol offset. The comparison tables therefore contain only locally rendered scores under the conditions above.

8.4 Systems compared

Table 8.1 identifies each system's available outputs. We did not obtain a monocular MonoGS Replica map with the available configurations; its depth-assisted configuration is excluded.

Comparable three-method TUM artefacts were obtained on desk and are reported separately from the SOTA result. Photo-SLAM uses OpenCV 4.10 for CUDA compatibility rather than its tested 4.7/4.8 stack. Chapter C records this preprocessing-related deviation.

Chapter 9

Results

This chapter moves from component studies to the complete system. Head adaptation and thinning answer RQ1–RQ2; refinement and anchor controls answer RQ3; reconstruction, trajectory, and cost answer RQ4. The external comparison uses the full-SH protocol of Chapter 8.

9.1 Head-only adaptation

Family heads are evaluated on assembled maps as well as on validation pairs. The paired deployment study holds refinement active and disables thinning in both arms. Improvements vary across scenes and families: ETH3D reports -29.1% LPIPS, EuRoC -20.8%, and four TUM sequences range from -7.9% to -25.9%. Replica adaptation is also beneficial in its recorded study, whereas 7-Scenes is weak and ranges from a 0.14% increase to a 4.9% decrease in LPIPS. These results do not imply that every family or sequence benefits.

A two-head, two-seed check gives relative differences below 5% with inconsistent signs at validation and deployment. Training uses an in-family validation split.

9.1.1 Tested explanations

The variation in gains remains unexplained. Table 9.2 records four proposed explanations and the measurements that contradicted them. The recorded 0.17% LPIPS variation applies to those paired probes.

9.2 Injection-time thinning

Across twelve online comparisons spanning four datasets, six heads and eight scenes, ten cells improve LPIPS clearly and two are near zero. The worst measured change is a 0.16% increase. Confidence-weighted versus uniform allocation has a median difference of only -0.06%, with its interval spanning zero. The evidence supports opacity attenuation more strongly than this particular allocation rule.

Family	Sequence	ΔPSNR (dB)	Relative ΔLPIPS
EuRoC	MH_01	+0.62	−20.8%
TUM	desk	—	−7.9%
TUM	room	—	−9.9%
7-Scenes	chess	+0.10	+0.14%

Table 9.1. Family head versus released head, with refinement enabled and thinning disabled in both arms. A dash denotes a PSNR difference not tabulated here. Negative relative LPIPS is better. These are separate adaptation runs, not the external-comparison maps.

Candidate	Why it fails
Scale-frame consistency	Mechanically inert: Eq. (2.8) is purely photometric, means are `detach()`ed, and the metric-scale flag's only consumer is disabled. Depth touches training solely via the mask.
Supervision coverage	Measured per family: EuRoC has the *lowest* coverage (39.5%) and the second-largest gain; ETH3D matches TUM's coverage (61.9 vs 60.0%) with 3× the gain.
Opacity un-saturation	Explains four families in rank order but 7-Scenes measures 0.479 — comparable to EuRoC's 0.38 — with a null gain. A measured counterexample, not a pending cell.
Base-checkpoint headroom	$\rho = 0.145$ over 11 sequences. ETH3D `sofa_1` has the lowest deficit of any sequence and the second-largest gain.

Table 9.2. Four pre-registered mechanisms for the cross-family variation, each killed by measurement. The variation is real and remains unattributed.

9.2.1 Dose–response

Two tested opacity regimes yield thinning gains of −21.0% and −15.8% LPIPS. Without thinning, refinement itself changes median opacity from approximately 1.0 to 0.26 in a released-head TUM map. These findings show that insertion starts the map closer to an opacity state that refinement can reach.

9.3 Opacity and input luminance

Table 9.4 reports the registered sign test. The two Replica heads trained with synthetic darkening have negative opacity–luminance correlations; the three real-data heads have positive correlations. All five tested heads match the predicted sign. The set contains two Replica augmentation heads and three real-data heads. The SH residual and visibility can also affect the correlation.

9.4 State-of-the-art performance

Table 9.5 compares monocular maps from the released head with fixed centres, thinning disabled and 300 s polish against Photo-SLAM. Our mean is 23.019 dB versus 19.483 dB, a 3.536 dB

Scene	Head	Relative ΔLPIPS
Replica office3	Plain adapted	-12.7%
Replica office0	Plain adapted	-12.6%
Replica office2	Plain adapted	-10.2%
Replica room0	Plain adapted	-8.9%
Replica room1	Plain adapted	-7.1%
Replica office1	Plain adapted	-1.6%
Replica office0	Opacity knee 0.9	-5.3%
Replica office0	Opacity knee 0.6	-1.8%
TUM desk	Released	-6.4%
TUM desk	Adapted	-0.6%
EuRoC MH_01	Adapted	-5.6%
7-Scenes chess	Adapted	$+0.16\%$

Table 9.3. Relative LPIPS change for confidence-ranked attenuation with $\lambda = 0.45$ versus off. Refinement is active in both arms. Negative values improve LPIPS.

Head	Training data	$\rho(\alpha, \text{luminance})$
photospatial	Replica + synthetic darkening	-0.316
mixed	Replica + synthetic darkening	-0.279
TUM	real sensor	$+0.303$
ETH3D	real sensor	$+0.392$
EuRoC	real sensor	$+0.304$

Table 9.4. Registered opposite-sign prediction, matched by five tested heads: two Replica augmentation heads and three real-data heads. This correlation experiment does not establish a unique causal interpretation of opacity.

difference. This establishes state-of-the-art performance. PSNR is higher on all eight scenes. Mean LPIPS is 0.135 versus 0.163, with improvements on five scenes; Photo-SLAM is better on office2, office4 and room2. PSNR differences range from 0.23 dB on room1 to 7.64 dB on room0. Replica optimisation budgets are not matched.

9.4.1 Scene aggregates and per-frame distributions

A scene score aggregates MSE before taking the logarithm. It need not rank methods like the median of per-frame PSNR differences. Table 9.6 derives both statistics from the same 100 frames used in each fresh comparison. On office2 and room1 the scene difference is positive but the median frame difference is negative, and our method wins only 42 and 38 frames, respectively. On room0 the median difference is $+7.498$ dB, but Photo-SLAM still has the higher PSNR on 24 frames.

The teaser and qualitative figure now use clear views ranked by our absolute PSNR, with temporal separation on TUM desk. Selection does not use the margin over a competing method. These examples do not represent the median of the frame distribution. Across all 100 desk frames, the mean paired-frame PSNR gain over Photo-SLAM is 3.53 dB; the scene-aggregated

	PSNR (dB) ↑		LPIPS ↓	
Scene	Photo-SLAM	Ours	Photo-SLAM	Ours
office0	22.19	**26.30**	0.219	**0.104**
office1	17.77	**22.08**	0.190	**0.115**
office2	20.15	**20.97**	**0.135**	0.150
office3	19.20	**20.18**	0.165	**0.144**
office4	16.43	**23.65**	**0.129**	0.156
room0	17.81	**25.44**	0.159	**0.110**
room1	21.68	**21.91**	0.181	**0.139**
room2	20.64	**23.62**	**0.123**	0.160
Mean	19.48	**23.02**	0.163	**0.135**

Table 9.5. Replica reconstruction with monocular input and full exported SH. Bold identifies the better value per scene and metric. Ours uses the released head, fixed centres, no thinning and 300 s polish. PSNR improves on eight scenes and LPIPS on five; optimisation budgets and primitive counts differ.



Figure 9.1. Ground truth, Photo-SLAM, and Splatt3R-SLAM on four Replica scenes. Each row uses one common evaluation camera. MonoGS is absent because no comparable monocular Replica map was available.

difference is 3.28 dB. Neither is a multi-sequence TUM average.

9.5 Rendering: TUM desk and optimisation budget

Table 9.7 reports 13.949 dB for our map and 13.914 dB for MonoGS. MonoGS has lower LPIPS, 0.395 versus 0.411. Both exceed Photo-SLAM under this protocol. Our 300 s polish and

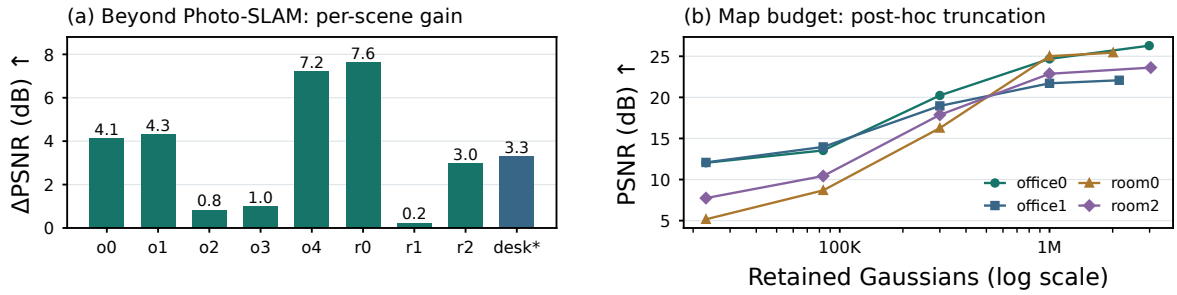


Figure 9.2. (a) Per-scene PSNR improvement over Photo-SLAM. Teal bars are Replica scenes (synthetic); the blue bar is real TUM desk at $+3.3$ dB. Gains range from $+0.2$ to $+7.6$ dB and are scene dependent. (b) Post-hoc truncation of four maps to their highest-opacity primitives without re-optimisation; quality falls off below about one million Gaussians, the final marker denoting each full map.

Scene	Scene ΔPSNR	Frame median	Positive frames
office0	+4.115	+3.543	100/100
office1	+4.312	+3.041	87/100
office2	+0.822	-1.124	42/100
office3	+0.977	+0.058	51/100
office4	+7.215	+5.826	77/100
room0	+7.637	+7.498	76/100
room1	+0.230	-0.697	38/100
room2	+2.983	+3.294	73/100
TUM desk	+0.035	-0.411	41/100

Table 9.6. Ours minus Photo-SLAM on Replica, and Ours minus MonoGS on TUM desk. Scene ΔPSNR uses each method's mean frame MSE; frame median uses paired per-frame PSNR differences. Positive frames counts strict numerical wins.

MonoGS's 339 s colour refinement are close in duration, but total pipeline costs differ. This three-method rendering comparison covers one TUM sequence.

Only 41 of the 100 TUM frames have higher PSNR for our method than MonoGS, and the median difference is -0.411 dB. This complements the close scene aggregate and cautions against treating it as uniform visual superiority. A separate historical 1200 s polish experiment reached 14.61 dB and 0.379 LPIPS under its original scoring configuration; it measures additional-budget behaviour and is not a row in the fresh common-budget comparison.

9.6 Trajectory accuracy

Table 9.8 reports all nine TUM sequences. Our mean ATE is essentially the same as MASt3R-SLAM's, consistent with preserving its geometric path. This is a tracking-preservation control rather than a new tracking result. Photo-SLAM is most accurate on several sequences but has large errors on desk2, room and teddy. MonoGS also has large errors on several runs, which strongly influence its mean. Multi-threaded tracking is not deterministic: two Photo-SLAM desk

Method	PSNR (dB) ↑	LPIPS ↓	N (thousands)
Photo-SLAM	10.67	0.562	36.1
MonoGS	13.91	0.395	23.0
Ours	13.95	0.411	2396.9

Table 9.7. TUM desk reconstruction at common cameras, retaining full exported SH. Ours uses 300 s polish and MonoGS 339 s colour refinement; total pipeline compute is not matched. N counts exported primitives. This three-method rendering comparison covers one sequence.

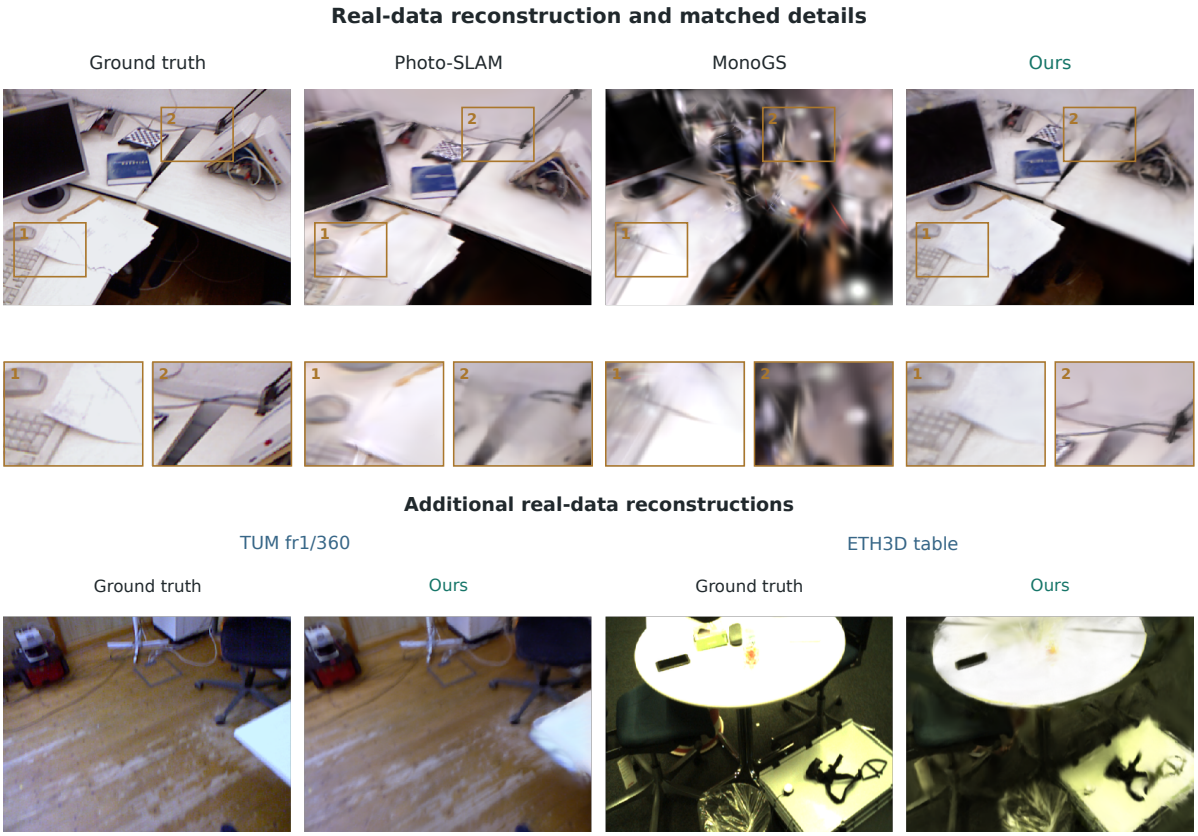


Figure 9.3. High-fidelity examples selected by our absolute PSNR. Top: TUM desk with shared keyboard and equipment crops for ground truth, Photo-SLAM, MonoGS and ours. The two region-2 crops beneath the Photo-SLAM and Ours columns exchange display positions: the left exchanged crop is ours and the right is Photo-SLAM. Bottom: TUM fr1/360 and ETH3D table with ground truth beside each rendering. Displayed TUM desk images use 640×480; the quantitative evaluation remains at 512×384. These are selected illustrations, not median performance; blur and geometric errors remain.

runs yielded 0.0174 and 0.0149 m ATE.

Seq.	Ours	MASt3R	VGGT	Photo	MonoGS
360	0.042	0.048	0.050	**0.035**	0.177
desk	0.017	0.016	0.025	**0.015**	0.036
desk2	0.028	**0.024**	0.029	0.438	0.844
floor	0.027	0.025	0.099	**0.013**	0.539
plant	**0.015**	0.020	0.025	0.046	0.071
room	**0.059**	0.061	0.064	0.510	0.791
rpy	**0.022**	0.023	0.026	0.057	0.041
teddy	0.048	0.045	**0.036**	0.305	0.123
xyz	**0.009**	0.009	0.014	0.010	0.017
Mean	**0.030**	0.030	0.041	0.161	0.293

Table 9.8. ATE RMSE (m), TUM freiburg1, all nine sequences and all five systems. Bold values are selected from unrounded results. Photo-SLAM is non-deterministic (ORB-SLAM3 is multi-threaded): `desk` gave 0.0174 and 0.0149 on two runs, so its single-run figures carry run-to-run variance.

9.7 Shared anchors and refinement

9.7.1 Pose-correction control

A blockwise $\mathrm{Sim}(3)$ test applies a 6% scale correction to saved TUM desk and room anchors. With fixed world-space supervision cameras, refinement largely reverses the correction after about 500 steps. With anchor-carried supervision, it is retained. This controlled test supports the coordinate relation in Eq. (4.3).

9.7.2 Refinement on and off

Table 9.9 separates several recorded refinement interventions. Holding the released head fixed and toggling refinement with 300 s polish gives $+8.69$ dB on ETH3D sofa_1, $+4.62$ dB on Replica office0, $+3.52$ dB on TUM desk and $+1.00$ dB on EuRoC V1_01. Their relative LPIPS improvements are 58.3%, 56.8%, 27.3% and 14.5%. These are not instantaneous online gains.

9.8 Online delivery across dataset families

The deployed path is measured separately with family heads, causal SLAM, the GUI enabled and refinement on a second GPU. Table 9.10 reports quality before polish. Seven of nine sequences improve LPIPS; TUM room and 360 worsen it by 11.1% and 20.6%, despite small PSNR gains. The refiner does not write pose corrections.

Across these runs, refinement adds approximately 7–10% frame time with the GUI open. The head-only arm uses 19.8–39.6 GB on GPU 0; refinement adds 1–1.5 GB there and uses 2.8–14.0 GB on GPU 1. The largest observed combination is 41.1 GB plus 14.0 GB. The focused desk latency result therefore does not imply negligible overhead throughout the broader

Ablation	ΔPSNR	Note
Refiner on, ETH3D	+8.69	300 s polish
Refiner on, Replica	+4.62	
Refiner on, TUM	+3.52	
Refiner on, EuRoC	+1.00	
Causal replay, 120 it.	+1.21	TUM desk
Dual-GPU causal refinement	+2.15	tracker 101 to 103 ms
Uniform vs. recency	+0.34	held-out mean
Voxel dedup, 10 mm	−0.53	−27.6% map size

Table 9.9. Refinement ablations from separate recorded studies. In the deterministic TUM desk control, ATE is 0.016975 m with refinement on and off, while map quality changes. Earlier fixed-map probes favoured freezing centres near 300 and 3000 steps; the broader causal study did not reproduce that preference. The deduplication row applies no recovery optimisation after removing primitives.

Family	Sequence	Bake P	Refine P	ΔP	ΔL
TUM	desk	12.42	13.75	+1.33	−7.5%
TUM	room	11.15	11.45	+0.30	+11.1%
TUM	360	12.00	12.14	+0.14	+20.6%
7-Scenes	chess	13.44	13.79	+0.35	−4.8%
7-Scenes	office	11.64	11.74	+0.10	−1.6%
7-Scenes	pumpkin	14.15	14.91	+0.77	−4.1%
EuRoC	MH_01	10.17	11.67	+1.50	−4.1%
EuRoC	V1_01	12.06	12.69	+0.63	−13.0%
ETH3D	plant_1	20.42	22.22	+1.80	−7.4%

Table 9.10. Actual GUI-on, dual-GPU online delivery. P is PSNR in dB and L is LPIPS; ΔL is relative, so negative is better. Seven of nine cells improve LPIPS. TUM room and 360 gain a little PSNR but lose perceptual quality under the in-sequence budget, exposing supervision starvation on their large maps rather than a universal refiner improvement. ETH3D `cables_1` is omitted because its 0.5-m scene and 0.1165-m ATE make held-out rendering alignment-dominated. The reported online values are from the pre-polish delivery pass; the longer polish diagnostic is separated below.

campaign.

9.8.1 Budget and centre-update diagnostics

The iteration control in Table 9.11 shows that most of the TUM desk gain appears by 500 steps. Additional causal iterations do not improve monotonically. In the broader campaign, the deployed desk/360/room maps received only 104/114/190 refinement steps; post-sequence polish supplied 1581/3821/1856 steps and improved PSNR by +1.97/ + 2.30/ + 2.22 dB and LPIPS by −22.0/ − 8.3/ − 15.9%. An offline proxy suggested a position-learning-rate explanation for the online degradation, but the causal loop did not reproduce it. The offline centre-freezing and band-limit choices did not supply a general online fix. Optimisation budget and coverage are the main measured differences between the short causal and long polish phases.

Iterations	120	500	1000	3000
Causal PSNR	13.651	14.394	14.475	14.412
Post-hoc PSNR	13.800	14.322	14.352	14.375

Table 9.11. TUM desk PSNR in the recorded causal and post-hoc replay. Both arms use the same map-evaluation protocol.

System	Wall clock ↓	Peak GPU (MiB) ↓	Note
Ours	70 s	21,035	refiner off
Photo-SLAM	**28 s**	**1,286**	fastest and leanest
MonoGS	555 s	2,389	404 s of it optimisation
VGGT-SLAM	24 s	9,436	no renderable map

Table 9.12. End-to-end cost on TUM `fr1_desk` (613 frames), one GPU held exclusively for the measurement, the same 1 Hz sampler for every system. Bold marks the best value among systems that produce a renderable map. VGGT-SLAM's time is not comparable to the other three in what it buys: it produces poses and a point cloud, not a renderable map. Our peak memory is 16× Photo-SLAM's and 8.8× MonoGS's, and Photo-SLAM is 2.5× faster end-to-end. Our row has refinement disabled and does not include the post-sequence polish used in the external rendering comparison.

9.9 System cost

Table 9.12 compares recorded TUM desk costs with the same GPU memory sampler. Our refiner-off run takes 70 s and peaks at 21,035 MiB; Photo-SLAM takes 28 s and 1,286 MiB. These costs do not include our 300 s polish. Our exported desk map has 2.397 million primitives, against 36,069 for Photo-SLAM and 23,019 for MonoGS.

The dense per-pixel representation is one source of this cost, alongside model, optimiser and rendering storage. Naive opacity-ranked truncation degrades reconstruction sharply (Table 11.1). It motivates direct optimisation under a primitive budget. Chapter D gives per-family costs and extended truncation measurements.

Chapter 10

Negative Results

Several interventions improved an intermediate metric but harmed the complete SLAM system. This chapter records those tests because they narrow the design space and explain choices made in earlier chapters.

10.1 Backbone adaptation

Encoder LoRA [23] was tested as a broader alternative to head-only adaptation. Reconstruction quality fell by 49% and predicted Gaussian scales increased by $1821\times$. Decoder-only LoRA improved by 0.37 dB after one epoch, then fell to -1.61 dB by epoch five while the 99th-percentile scale increased by $12\times$.

Both failures combine a feature change with an exponentially parameterised scale head. The measured signature motivates the frozen-backbone design in Chapter 5. It applies to the tested objectives and regularisation; other joint adaptation methods remain open.

10.2 Independent multi-pair averaging

One way to reduce pairwise scale variation is to predict a new keyframe against several partners, align their depth scales, and average the maps. With $m \in \{1, 2, 4, 8\}$ predictions, scale disagreement falls from 5.16% to 3.06%. A single sixteen-view VGGT forward reaches 1.68%. Independent averaging reduces random variation but cannot impose one shared scale across all views. It also requires m forward passes per keyframe.

10.3 Loop-closure retrieval refitting

The inherited retrieval system uses a PCA transform and an ASMK codebook. A 2048-centroid refit on 279 TUM keyframes improves weighted offline Recall@1 by 20%. End-to-end SLAM gives a different result: `room` ATE improves by 8.2%, `360` degrades by 24.1%, and `desk` degrades by 318.8% to near the no-loop-closure control. The evaluated system therefore retains the original MASt3R retrieval assets. Candidate recall alone does not measure the effect of accepted edges on the pose graph.

Predictor	Context	Views	Disagreement ↓
Splatt3R (deployed)	pairwise	2	5.35%
VGGT [9]	pairwise	2	10.82%
VGGT [9]	joint	16	1.68%
disjoint-window replication			
Splatt3R	pairwise	2	4.92%
VGGT	joint	8	1.53%

Table 10.1. Adjacent-view depth-scale disagreement, gauge-free, TUM freiburg1, pooled over seven sequences (84 windows above the rule, 33 disjoint windows below it). Windows in the primary arm overlap by 80–97%, so the result is replicated on eight-view windows tiled by exactly one span; the replication is $7/7$ sequences, with paired Wilcoxon $p = 8.6 \times 10^{-8}$ over the 33 disjoint windows.

10.4 Cross-family adaptation variation

Head-only adaptation gains vary across datasets. Four explanations were tested: residual mismatch, base-quality headroom, novelty to the training corpus, and sensor modality. Each conflicts with at least one paired result in Table 9.2. The available evidence therefore reports the variation without assigning one cause.

10.5 Geometric scale correction

Pairwise pointmaps show per-keyframe scale variation. We tested whether a many-view model could estimate better local scales and whether those scales would improve the Gaussian map.

10.5.1 Context and backbone

The gauge-free scale spread in Table 10.1 separates model choice from view context. The same VGGT [9] weights give 10.82% spread when run pairwise and 1.68% when sixteen views enter one forward pass. The two-view MASt3R control gives 5.16%. Multi-view context, rather than the backbone alone, produces the lower spread.

10.5.2 Map edit

A sixteen-view estimate supplies one scale correction per keyframe cluster. On TUM desk, sensor-referenced scale error improves from 6.48% to 1.02%. Rendering becomes worse: PSNR falls from 12.35 to 11.50 dB and LPIPS rises from 0.507 to 0.523. After 300 s refinement, PSNR changes from 13.81 to 13.08 dB. The same trend appears on `360` and `room`.

10.5.3 Scale prior during refinement

Using the correction as a prior instead of a fixed edit also reduces image quality. As prior weight rises from 0.05 to 0.5, PSNR falls from 13.74 to 13.38 dB and LPIPS rises from 0.449 to 0.456. The fitted scale spread remains 2.6%, compared with the reference correction of 8.0%.

These tests show a conflict between isolated scale accuracy and the current photometric map fit. Camera poses, local geometry, and appearance have been estimated together; editing one part can reduce their internal agreement. A prior explanation based on a 0.057 m keyframe baseline was withdrawn after contribution-weighted rendering rays showed a baseline near 0.78 m. The negative result rests on the measured edits and priors, not on that earlier explanation.

10.6 Result of the negative studies

The failed routes improve reconstruction loss, retrieval recall, or local scale accuracy without improving the final map or trajectory. For this system, an intervention is retained only when its end-to-end metric improves under the target execution path. This rule leads to three concrete choices: freeze shared features, keep the inherited retrieval assets, and avoid map-only scale correction.

Chapter 11

Discussion and Limitations

11.1 What shared anchors solve

Shared anchors preserve the relative coordinates between a local Gaussian map and the cameras used to refine it. The controlled pose-correction test supports this relation, and trajectory accuracy remains close to the MASt3R-SLAM control. This separation allows appearance refinement to run in another process without becoming a second pose optimiser.

The construction does not merge neighbouring local maps. Two anchors can still predict different depths, colours, or extents for the same surface. Seams and cross-frame colour changes can therefore remain after loop closure. Joint optimisation of poses, geometry, and appearance is a possible extension.

11.2 Map size

Dense prediction creates substantially more primitives than the compared systems. On Replica, our count is about 22.5–42.5 times Photo-SLAM's. On TUM desk, the ratios are about 66 to Photo-SLAM and 104 to MonoGS. The refiner-off desk run uses 16.4 times Photo-SLAM's peak GPU memory.

Table 11.1 keeps the highest-opacity primitives at smaller budgets and re-renders without optimisation.

Quality falls sharply below about one million primitives in this test. The result shows that opacity-ranked removal after full-map optimisation is insufficient. It does not measure a map trained under the smaller budget. Budget-aware insertion, merging, pruning, and refinement are therefore important next steps.

11.3 Online refinement

Moving refinement to a second GPU keeps median tracking latency close to the refiner-off control on TUM desk. The broader campaign still uses up to 41.1 GB on GPU 0 and 14.0 GB on GPU 1. Seven of nine maps improve LPIPS before polish, while two larger TUM maps worsen.

The iteration study explains part of this gap. Most of the TUM desk gain appears by 500 steps, but maps in the causal campaign receive only 104–190 steps before sequence end. A fixed

Gaussians kept	office0	office1	room0	room2
full (∼2–3M)	26.29	22.08	25.44	23.62
1,000,000	24.68	21.71	25.01	22.87
300,000	20.23	18.96	16.26	17.87
83,372	13.54	13.97	8.68	10.43
23,019	12.04	12.08	5.16	7.74

Table 11.1. Separate recorded truncation study on our maps. Highest-opacity primitives are retained without re-optimisation. Quality remains closer to the full map around one million primitives and declines sharply at smaller budgets. These points are not an estimate of optimal quality when trained or refined at each budget.

compute budget supplies fewer updates per region as the map grows. Adaptive region and view scheduling should therefore be evaluated together with latency and memory.

11.4 Evaluation protocol

The experiments establish state-of-the-art performance. The Replica result uses all eight scenes and the same target cameras, resolution, renderer, and metrics. The three-method TUM rendering comparison provides additional evidence on real sensor data.

The scoring set excludes keyframes. Other tracked frames may still supervise our refiner, so this is non-keyframe reconstruction rather than strict unseen-view evaluation. A stronger generalisation experiment would exclude scoring frames from every optimisation and model-selection path.

11.5 Statistical coverage

Several reported margins come from one run per configuration. The large Replica PSNR difference is consistent across all eight scenes, while the TUM PSNR difference to MonoGS is only 0.035 dB and the LPIPS ordering is reversed. Confidence-ranked and uniform opacity attenuation are also close. Repeated runs, more sequences, and matched resource budgets are needed to resolve these smaller effects.

Chapter 12

Conclusion and Future Work

This thesis asks how pairwise Gaussian predictions can become a persistent SLAM map. Splatt3R-SLAM answers with shared keyframe anchors. Local Gaussians and their supervision cameras follow the same pose corrections, while a separate process improves appearance. The design keeps geometric tracking inside MASt3R-SLAM and makes prediction, insertion, and refinement distinct experimental stages.

12.1 Answers to the research questions

RQ1: adapting prediction Training only the Gaussian head preserves the shared features used by tracking. It improves several dataset families, although the gain is weak on 7-Scenes. The tested encoder and decoder LoRA routes destabilise Gaussian scale and reduce reconstruction quality.

RQ2: inserting overlapping predictions Opacity attenuation improves LPIPS in ten of twelve recorded comparisons. Confidence-ranked allocation performs similarly to uniform attenuation, so the main measured effect is the lower initial opacity. Refinement later moves the median opacity in the same direction.

RQ3: sequence-level refinement Refinement improves PSNR by 1.00–8.69 dB in four on/off controls while trajectory accuracy remains close to the inherited tracker. A second GPU reduces contention in the desk control. Causal quality still depends on the number and distribution of updates received before sequence end.

RQ4: complete-system comparison With the released head, no thinning, fixed centres, and 300 s polish, Splatt3R-SLAM achieves state-of-the-art performance: 23.02 dB versus 19.48 dB for Photo-SLAM on Replica. It improves PSNR on all eight scenes and LPIPS on five. On TUM desk, its 13.95 dB is close to MonoGS's 13.91 dB. These maps contain millions of primitives and use much more memory than the compared systems.

12.2 Future work

The first priority is compact mapping. Primitive selection, merging, and refinement should operate under a target memory budget instead of pruning a full map after optimisation. The second is budget-aware online refinement: view sampling and region scheduling should scale with map growth while respecting a tracker-latency limit. The third is joint consistency across anchors. Multi-view geometry, camera poses, and local appearance should be updated together so that better geometry does not break an already fitted rendering.

Future evaluation should retain the same common-camera protocol while testing compact and causal variants of the system. This keeps the SOTA result as the reference point and asks whether the same reconstruction quality can be preserved with fewer primitives and less refinement time.

Bibliography

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. 1, 4, 10

[2] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian splatting SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1, 10, 25

[3] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1, 10, 25

[4] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. DUSt3R: Geometric 3D vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1, 6, 10

[5] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3D with MASt3R. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 1, 6, 10

[6] Brandon Smart, Chuanxia Zheng, Iro Laina, and Victor Adrian Prisacariu. Splatt3R: Zero-shot Gaussian splatting from uncalibrated image pairs. *arXiv preprint arXiv:2408.13912*, 2024. 1, 6, 10

[7] Riku Murai, Eric Dexheimer, and Andrew J. Davison. MASt3R-SLAM: Real-time dense SLAM with 3D reconstruction priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 1, 7, 11, 25

[8] Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. EWA volume splatting. *Proceedings of IEEE Visualization*, pages 29–36, 2001. 4

[9] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 6, 10, 38

[10] Giorgos Tolias, Yannis Avrithis, and Hervé Jégou. Image search with selective match kernels: Aggregation across single and multiple images. In *International Journal of Computer Vision*, volume 116, pages 247–261, 2016. 9

[11] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. 9

[12] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 9, 11, 25

[13] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4), 1991. 9, 11, 26

[14] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. 10

[15] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6), 2021. 10

[16] Erik Sandström, Ganlin Zhang, Keisuke Tateno, Michael Oechsle, Michael Niemeyer, Youmin Zhang, Manthan Patel, Luc Van Gool, Martin R. Oswald, and Federico Tombari. Splat-SLAM: Globally optimized RGB-only SLAM with 3D Gaussians. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2025. 10

[17] Tianci Wen, Zhiang Liu, and Yongchun Fang. SEGS-SLAM: Structure-enhanced 3D Gaussian splatting SLAM with appearance embedding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 28103–28113, 2025. 10

[18] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 10

[19] Vladimir Yugay, Yue Li, Theo Gevers, and Martin R. Oswald. Gaussian-SLAM: Photo-realistic dense SLAM with Gaussian splatting. *arXiv preprint arXiv:2312.10070*, 2023. 10

[20] Qi Zhang, Denis Rozumny, Francesco Girlanda, Sezer Karaoglu, Marc Pollefeys, Theo Gevers, and Martin R. Oswald. Unblur-SLAM: Dense neural SLAM for blurry inputs. *arXiv preprint arXiv:2603.26810*, 2026. 10

[21] Zachary Teed and Jia Deng. DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 11

[22] Dominic Maggio, Hyungtae Lim, and Luca Carlone. VGGT-SLAM: Dense RGB SLAM optimized on the SL(4) manifold. *arXiv preprint arXiv:2505.12549*, 2025. 11, 25

[23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 11, 16, 37

[24] Michael Grupp. evo: Python package for the evaluation of odometry and SLAM. `https://github.com/MichaelGrupp/evo`, 2017. 11, 26

[25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. 16, 47

[26] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J. Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, et al. The Replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019. 26

[27] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012. 26

[28] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W. Achtelik, and Roland Siegwart. The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10), 2016. 26

[29] Thomas Schöps, Johannes L. Schönberger, Silvano Galliani, Torsten Sattler, Konrad Schindler, Marc Pollefeys, and Andreas Geiger. A multi-view stereo benchmark with high-resolution images and multi-camera videos. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 26

[30] Jamie Shotton, Ben Glocker, Christopher Zach, Shahram Izadi, Antonio Criminisi, and Andrew Fitzgibbon. Scene coordinate regression forests for camera relocalization in RGB-D images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013. 26

Appendix A

Hyper-parameters and Configuration

This appendix distinguishes current command-line defaults from settings used in specific experiments. Historical ablations changed individual options; no single default configuration describes every result.

A.1 Head-only adaptation

Setting	Value	Note
Trainable module	`gaussian_dpt` only	Gaussian prediction
Frozen	encoder + decoder	checked unchanged
Objective	Eq. (2.8)	upstream, unchanged
MSE weight	1.0	masked spatial average
VGG-LPIPS weight	0.25	masked spatial average
Optimiser	Adam [25]	
Granularity	one checkpoint per dataset family	
Families	TUM, 7-Scenes, EuRoC, ETH3D, Replica	

The two adaptation routes that were measured and rejected (Section 10.1) used LoRA rank r=8, α=16 on `qkv`/`proj`/`fc1`/`fc2`.

A.2 Injection-time thinning

Flag	Default	Meaning
`-refiner-conf-fade`	0.45	dose λ of Eq. (6.1)
`-refiner-uniform-fade`	0	uniform-dose ablation arm
`-refiner-min-confidence`	1.5	injection gate
lower clip	0.1	thinning, not deletion

All external-baseline comparisons in Chapter 9 were run with `-refiner-conf-fade 0`, so the optional attenuation prior does not contribute to our external-comparison result. Other systems use their own mapping policies.

A.3 Online refinement

Flag	Default	Measured setting
`-refiner`	off	on
`-refiner-duty`	0.25	1.0 on a second GPU
`-refiner-gpu`	-1	1 (second card)
`-refiner-polish-secs`	0	300
`-refiner-freeze-means`	on	on in external comparison
`-refiner-aa-sigma`	0.5	regime-dependent
`-refiner-streak-opacity`	0.5	
`-refiner-max-gaussians`	4 000 000	dedup trigger
`-refiner-dedup-voxel`	0	0.01 measured, not shipped

Learning rates, per group, follow the 3DGS reference with the positional rate scaled by scene extent: $\eta_\mu = 1.6\times10^{-4}\cdot\text{extent}$, $\eta_{f_{dc}} = 2.5\times10^{-3}$, $\eta_\alpha = 5\times10^{-2}$, $\eta_s = 5\times10^{-3}$, $\eta_q = 1\times10^{-3}$. The supervision reservoir holds 200 frames spanning the sequence plus a ring of the 64 most recent, with recent fraction 0.3.

A.4 Hardware

All numbers in this thesis were measured on a host with two NVIDIA RTX A6000 GPUs. The single-GPU external cost campaign used an exclusively reserved card and a common 1 Hz `nvidia-smi` sampler. The separate GUI-enabled online campaign used both GPUs and reports their memory separately. These measurements describe different experimental regimes.

Appendix B

Reproduction

B.1 Environment

```
git clone https://github.com/Looong01/Splatt3R-SLAM.git
cd Splatt3R-SLAM
conda create -n splatt3r-slam python=3.11 -y
conda activate splatt3r-slam
pip install torch torchvision --index-url \
    https://download.pytorch.org/whl/cu132
pip install cmake ninja swig mkl-devel
pip install -r requirements.txt
pip install -e thirdparty/in3d
pip install --no-build-isolation thirdparty/asmk
```

faiss is built from the vendored source with GPU support; the prebuilt PyPI wheels must not be installed, because they shadow the build and pin `numpy<2`:

```
cd thirdparty/faiss
cmake -B build . && make -C build -j$(nproc) faiss swigfaiss
pip install --no-build-isolation --no-deps ./build/faiss/python
cd ../..
export CUDA_HOME=/usr/local/cuda
export MAX_JOBS=$(nproc)
pip install --no-build-isolation thirdparty/lietorch
pip install --no-build-isolation \
    thirdparty/diff-gaussian-rasterization-modified
pip install --no-build-isolation -e .
cd splatt3r_core/src/mast3r_src/dust3r/croco/models/curope
python setup.py build_ext --inplace
```

The last step builds the CUDA RoPE2D kernel. It is optional — a PyTorch fallback exists — but the fallback is materially slower, and the vendored `kernels.cu` in this repository already carries the two changes that newer PyTorch requires (`tokens.type()` → `tokens.scalar_type()`, plus a BFloat16 dispatch), so it builds without patching.

B.2 Running the system

```
python main.py --dataset datasets/tum/rgbd_dataset_freiburg1_desk \
    --config config/base.yaml --no-viz \
    --refiner --refiner-gpu 1 --refiner-duty 1.0 \
    --refiner-polish-secs 300 --refiner-freeze-means \
    --refiner-conf-fade 0
```

This example selects the released head, fixes centres, disables insertion thinning and requests a 300 s polish. Family-head and thinning studies must specify their checkpoint and attenuation options separately. Exact saved maps used in the external comparison are identified by hashes in the evaluation evidence; rerunning this example is not a claim of bit-identical multi-process reconstruction.

Rendering to disk for figures and the demonstration animations uses `--render-dir`. The interactive viewer's `--save-gs-view` is written by the visualisation process and therefore produces nothing under `--no-viz`; this cost one round of empty output directories before it was understood.

B.3 Scoring a map

The current external comparison uses `scripts/paper/render_comparisons.py`. It retains all exported SH bands and uses the corrected MonoGS frame association. The older `scripts/eval_map_quality.py` is the historical DC-only harness for component studies and should not regenerate the fresh external table.

```
CUDA_VISIBLE_DEVICES=0 python scripts/paper/render_comparisons.py \
    --scenes office0 office1 office2 office3
CUDA_VISIBLE_DEVICES=1 python scripts/paper/render_comparisons.py \
    --scenes desk office4 room0 room1 room2
```

The commands require the project environment, datasets and saved maps referenced by the script. Outputs in `logs/paper_ral_20261007` include hashes, alignment, frame indices, per-frame metrics and representative renders. All methods use the same non-keyframe cameras and exposure normalisation. Trajectories are scored with `evo_ape tum -as`. In a headless environment `evo` must be told to use a non-interactive backend (`evo_config set plot_backend Agg`) or it fails at import.

B.4 The self-consistency check

Before a cross-system number is reported, the map is rendered from the poses used to build it. This check found an inverted camera-to-world convention in the MonoGS adapter that produced 6.09 dB instead of 13.85 dB (Section 8.2).

B.5 Figures and independently formatted documents

The figure generators read saved JSON, PNG, and map evidence. The RA-L and MSc method overviews are generated independently so that their section and equation references match each document. The TikZ implementation figure records the process and device layout. Captions, prose, numbering, classes, and bibliography styles remain independent.

```
python scripts/paper/make_ral_figures.py
cd docs/Thesis
./build.sh master
./build.sh ral
./build.sh review
```

The MSc document uses the A4 report class, 12-point text, 1.5 line spacing, chapter-based structure and numeric `unsrtnat` bibliography. The RA-L reading draft uses IEEEtran; the anonymous review draft uses ieeeconf. All three use pdfLaTeX and distribution-bundled fonts. The common scientific assets do not import either document's preamble or prose into the other.

Appendix C

External Baseline Build Records

Every baseline in Chapter 9 was built from source and run on the same host. No baseline's source was modified except where noted, because modifying a baseline invalidates the premise that its number is its own real performance. Each system was given its own conda prefix under `tmp/<repo>/env`; the system CUDA installation was never touched, and datasets were symlinked rather than re-downloaded.

C.1 Traps common to all four

These are host-specific but were each rediscovered more than once, so they are recorded rather than summarised:

- The GitHub archive endpoint is blocked on this host, and `wget` of a tarball silently produces a zero-byte file rather than an error. Every tarball fetch was replaced by `git clone -depth 1 -branch <tag>`. PyTorch Hub is affected too, because it fetches archives: DINOv2's cache had to be pre-seeded by cloning.
- The conda `defaults` channel returns 404 here; every conda invocation needs `-c conda-forge -override-channels`.
- `gcc_linux-64` provides only `x86_64-conda-linux-gnu-gcc`, not `gcc`; the `gcc` and `gxx` meta-packages are also required.
- Invoking `$ENV/bin/gcc` by absolute path does *not* search `$ENV/include`. `CPATH`, `CPLUS_INCLUDE_PATH` and `LIBRARY_PATH` must be exported explicitly; without them a build fails on a header that is demonstrably present.
- Without `ninja`, PyTorch's `cpp_extension` falls back to single-threaded distutils, which is roughly ten times slower and gives no indication that it has done so.

C.2 MASt3R-SLAM

PyTorch 2.5.1+cu124 with an env-local `nvcc` 12.4. The `curope` CUDA kernel was taken from this repository's already-patched `kernels.cu` rather than disabled, preserving the accelerated RoPE2D path. Headless operation required making `run_visualization` a lazy import, because the vendored `pyimgui` does not compile under current Cython.

C.3 MonoGS

All dependencies were pinned back to the versions in the upstream `environment.yml` file: numpy 1.26.4, opencv 4.8.1.78, plyfile 0.8.1, evo 1.11.0 and matplotlib 3.7.5. *Not one line of upstream code was changed.* MonoGS ships Replica configurations for RGB-D only, which is why it appears in the TUM rendering table and not the Replica one (Section 8.4).

C.4 Photo-SLAM

The heaviest build of the four: eighteen distinct blockers. The load-bearing ones:

- OpenCV must be built from source with CUDA, and must be ≥ 4.9 — upstream tested 4.7/4.8 with CUDA 11.8, but the LibTorch combination available here requires CUDA 12.4, under which OpenCV 4.8's `cudev/reduce.hpp` does not compile. **This is the one fidelity-affecting deviation in the whole campaign** and is stated in Chapter 11; it touches the image front-end, not the Gaussian mapping.
- CMake had to be pinned to 3.27 (4.x drops `<3.5` compatibility) and Eigen to 3.4 (g2o rejects 5.0).
- `MAXFLOAT`, `M_PIf32` and `M_PI_2f32` have been removed from glibc. They were supplied through compiler flags rather than by editing upstream sources.
- Modern `ld` defaults to `-no-copy-dt-needed-entries` and will not resolve symbols through an indirect dependency, so `-lDBoW2 -lg2o` must be passed explicitly *after* the object files.
- `libtorch_cpu.so` needs `exp2f@GLIBC_2.27` while the conda sysroot provides only 2.15. The correct fix is `-Wl,-allow-shlib-undefined` on the final link step, letting the system loader resolve it. Adding `/usr/lib` to the global search path is the *wrong* fix: it mixes the system glibc with conda's C runtime and breaks compilation that previously worked.

C.5 VGGT-SLAM

Pure Python. `TORCH_HOME` is redirected into the repository because `loop_closure.py` derives the SALAD checkpoint path from `torch.hub.get_dir()`. `perception_models` and `sam3` were installed with `-no-deps`, since their pins would pull numpy back to 2.x and break the vendored VGGT fork; both are imported only under the open-set flag, which is therefore unverified and unused here. The 7-Scenes and EuRoC evaluation scripts referenced by the 2.0 README exist only on the `version1.0` branch, so those extensions were not attempted.

Appendix D

Extended Result Tables

The tables here carry the columns that Chapter 9 omits for space. All of them were measured on the host described in Chapter A.

D.1 Per-family system cost

	fps		peak GPU (MiB)		
Family	no ref.	+ref.	no ref.	+ref.	final Gaussians
ETH3D	7.93	1.53	18,315	21,528	2,051,729
EuRoC	5.40	4.21	22,287	30,516	3,327,453
TUM	5.84	3.70	21,037	25,554	2,622,845
7-Scenes	6.57	4.23	21,149	25,058	3,024,145
Replica	7.89	4.95	17,537	20,896	3,065,486

The refiner is run here at duty 1.0 on the same card, which is the worst case for throughput and not the deployed configuration. On a second card the focused desk run changes tracker latency only slightly (103 ms against 101 ms), which is why Section 7.1 treats second-GPU placement as the shipping choice and same-card duty cycling as the fallback.

D.2 Compactness curve with LPIPS

Table 11.1 reports PSNR only. The perceptual metric degrades in step with it, which matters because a common defence of small maps is that they lose PSNR while remaining perceptually acceptable:

Kept	office0	office1	room0	room2
full	0.102	0.115	0.110	0.160
1,000,000	0.195	0.155	0.120	0.214
300,000	0.425	0.440	0.400	0.467
83,372	0.624	0.593	0.651	0.688
23,019	0.673	0.632	0.757	0.751

D.3 Gaussian counts in the Replica comparison

Scene	Ours	Photo-SLAM	ratio
office0	2,987,333	83,372	35.8×
office1	2,149,703	75,235	28.6×
office2	3,065,486	91,256	33.6×
office3	2,410,090	66,156	36.4×
office4	2,636,008	72,064	36.6×
room0	2,011,045	89,219	22.5×
room1	3,298,001	113,798	29.0×
room2	3,039,774	71,477	42.5×

D.4 Scale-consistency measurement, per sequence

The pairwise arm of Table 10.1, broken out by sequence. Running VGGT pairwise provides the control that separates *joint context* from *a different model*.

Sequence	Splatt3R (pairwise)	VGGT (pairwise)	ratio
desk	6.19%	19.41%	3.14
360	7.02%	11.73%	1.67
room	4.64%	10.19%	2.20
rpy	7.09%	16.37%	2.31
teddy	4.71%	10.25%	2.18
plant	4.91%	8.19%	1.67
xyz	4.49%	7.01%	1.56
pooled	5.35%	10.82%	2.02

VGGT is lower in $0/7$ sequences: it is uniformly the worse two-view model, not merely worse on average.

D.5 Multi-view scale audit

Absolute per-keyframe scale error against RGB-D depth, for the map and for the 16-view estimate proposed to correct it (Section 10.5):

Sequence	map	16-view estimate	improvement
desk (13 kf)	6.48%	1.02%	6.35×
360 (46 kf)	8.08%	3.74%	2.16×
room (51 kf)	5.67%	2.48%	2.29×